INTERVIEW ANSWERS — PART 4a: AZURE (70 Questions)

SECTION 9: AZURE — All 70 Questions Answered

Q1. Different types of Azure services

IaaS (Infrastructure as a Service): Azure Virtual Machines.
You manage: OS patches, runtime installation, security hardening.
Use when: full control needed, legacy software.

PaaS (Platform as a Service): Azure App Service, Azure SQL.
You manage: only your application code and data.
Azure manages: OS, runtime, patches, load balancer.

SaaS (Software as a Service): Microsoft 365, Dynamics CRM.
You manage: nothing — just open browser and use.

FaaS (Function as a Service): Azure Functions.
You manage: only your function code.
Azure manages: everything, scales to zero, billed per execution.

DBaaS: Cosmos DB, Azure SQL Database — fully managed DB.

Q2. Examples of IaaS, PaaS, SaaS

IaaS — Azure Virtual Machine:
You RDP/SSH into the VM. You install .NET 8 runtime yourself.
You configure IIS. You patch Windows monthly.
Full control, but maximum effort.

PaaS — Azure App Service:
You push a ZIP file or Docker image.
Azure handles OS, runtime, load balancer, patches, SSL.
You never touch a server.

SaaS — Microsoft 365 / Outlook:
Open browser, sign in, write email.
Microsoft manages every server, every patch, every upgrade.

Q3. What is FaaS?

Function as a Service = serverless compute model.

How it works:
1. You write only the function code (a single method).
2. You define a trigger (HTTP call, timer, queue message).
3. Azure provisions infrastructure, runs your code, tears it down.
4. You pay ONLY for the milliseconds your code actually executes.
5. When no requests arrive, zero servers run (scale to zero).

Key trade-off: Cold start — first request after idle period takes longer (~200ms for .NET) because Azure must provision the container.

Q4. What are Azure Functions?

Event-driven, serverless compute units in Azure.

Each Function has two parts:
TRIGGER: what event starts execution
- HTTP: becomes a REST API endpoint
- Timer: cron schedule (run every day at 8 AM)
- Service Bus: process each queue/topic message
- Blob Storage: fires when file is uploaded
- Event Hub: process streaming events

BINDINGS (optional): connect input/output data sources
- Input binding: read from Cosmos DB, Storage Table
- Output binding: write to Queue, Blob, SignalR

Scale: automatic — 0 instances when idle, 200+ under heavy load.

Q5. How do you deploy Azure Function Apps?

Option 1 — Visual Studio: Right-click project → Publish → Azure Function App.
Option 2 — Azure DevOps YAML pipeline:
DotNetCoreCLI publish → AzureFunctionApp@1 task.
Option 3 — GitHub Actions:
actions/checkout → setup-dotnet → dotnet publish → azure/functions-action.
Option 4 — Docker container:
Build image → Push to Azure Container Registry → deploy to Function App.
Option 5 — Azure CLI:
az functionapp deployment source config-zip --src app.zip

Q6. How do you deploy applications to cloud platforms?

CI/CD Pipeline approach (standard in production):

Code push to main branch
→ CI pipeline triggered automatically
→ Restore NuGet packages
→ Build (Release config)
→ Run unit tests + collect code coverage
→ Publish artifact (compiled app)
→ Deploy to Dev environment (automatic)
→ Run integration tests
→ Deploy to Staging (automatic or manual trigger)
→ Manual approval gate (manager/architect approves)
→ Deploy to Production
→ Smoke test production
→ Monitor App Insights for error spikes

Infrastructure created separately via Bicep/Terraform (IaC).

Q7. New features in recent .NET versions?

.NET 8 (LTS — November 2023, supported until 2026):
Native AOT: compile to native binary with no JIT runtime.
Result: app starts in milliseconds, tiny memory footprint.

Ideal for: Azure Functions, containers, microservices.
Primary constructors: class Person(string name) { } — shorter syntax.
Blazor Full Stack: server + WASM unified rendering model.
Performance: JSON serialisation 20% faster than .NET 7.

.NET 7 (November 2022):
Rate limiting middleware built into ASP.NET Core.
Output caching middleware (cache entire response).
Minimal API improvements (route groups, filters).
Regex source generation (no reflection, faster).

Q8. What is default namespace in .NET Core?

No implicit global namespace. Each file declares its own.

Traditional (with braces):

```
namespace MyApp.Services
{
    public class OrderService { }
}
```

File-scoped (modern C# 10+ — preferred):

```
namespace MyApp.Services;   // applies to entire file, no braces needed
public class OrderService { }
```

Project-level default: set in .csproj:
<RootNamespace>MyApp</RootNamespace>
Used by tooling (Visual Studio, dotnet new) for auto-generated code.

Q9. Explain CI/CD pipeline.

CI (Continuous Integration):
Every git push triggers automated build + unit tests.
If tests fail: team notified immediately, no broken code merged.
Goal: always have a working, releasable codebase on main branch.

CD (Continuous Deployment):
Every passing build on main is automatically deployed.
Stages: Dev (auto) → Staging (auto/manual) → Production (approval).

Benefits:
- Bugs caught minutes after introduction (not weeks later).
- Deployment is repeatable and reliable (not manual steps).
- Shorter release cycles (daily deploys vs monthly).
- Rollback is easy (redeploy previous artifact).

Q10. How do you perform deployment without downtime?

Deployment Slots (Azure App Service — recommended):
1. Deploy new version to "staging" slot (separate URL, live in Azure).
2. New version warms up, Azure runs health checks.
3. SWAP slots — production URL now points to new version.
4. Swap is atomic and instant — zero downtime.
5. Old version now on staging slot — rollback = swap back.

Kubernetes Rolling Update:
Deployment YAML: strategy: type: RollingUpdate
maxUnavailable: 1 (allow 1 pod down at a time)
maxSurge: 1 (allow 1 extra pod during update)
K8s replaces pods one at a time.
Readiness probe must pass on new pod before old pod is killed.

Q11. Blue-Green vs Canary deployment.

Blue-Green:
Two complete identical environments (Blue = live, Green = new version).
Deploy to Green, test it, switch ALL traffic at once (DNS/load balancer).
Rollback: switch traffic back to Blue (instant).
Cost: double infrastructure while both live.
Best for: want instant rollback, simple traffic switching.

Canary:
Keep Blue (production) running.
Route small % of traffic (5%) to new version (Canary).
Monitor error rates, latency, business metrics.
Gradually increase: 5% → 25% → 50% → 100%.
If problems: route all traffic back to Blue.
Best for: risk-averse releases, want real-world validation at small scale.

Q12. How do you troubleshoot production issues?

Step-by-step investigation:

1. App Insights → Failures tab.
   Find the exception with full stack trace and exact timestamp.

2. Transaction search by correlation ID.
   User gives you their request ID → trace that exact request.

3. Check Dependencies panel.
   Was it a slow DB query? External API timeout? Which dependency?

4. Log Analytics KQL query.
   Find pattern: did errors spike at specific time? Only from certain region?

5. Compare with deployment history.
   Did issue start right after a deploy? Check what changed.

6. Live Metrics stream.
   Real-time view — watch while reproducing the issue in staging.

Q13. Debugging strategies in live systems.

Structured logging with correlation ID:
Every request: log.LogInformation("Processing {OrderId} for {UserId}", orderId, userId);
Correlation ID in every log entry → search App Insights: where correlationId == "abc123"
See complete picture of one user request across all services.

Distributed tracing:
App Insights automatically traces calls across microservices.
One transaction view shows: API → DB call (12ms) → Service Bus publish (3ms).

Snapshot debugging:
When exception occurs in production, App Insights captures local variable values.
View the snapshot in Azure portal — see exact state at moment of crash.

Live Metrics:
Real-time 5-second delay view of requests/failures/dependencies.
Watch while someone reproduces the issue.

Q14. Azure App Service — how to deploy .NET app.

App Service = managed PaaS web hosting.
Azure manages: OS, patches, load balancer, SSL, runtime installation.

Steps:
1. Create App Service Plan.
   Defines pricing tier and compute (B1=1 core 1.75GB, P1v3=1 core 1.75GB premium).
2. Create App Service.
   Choose: .NET 8, OS (Linux preferred for cost), region.
3. Configure App Settings and Connection Strings in Azure Portal.
   These become environment variables at runtime (override appsettings.json).
4. Deploy via:
   - VS Publish wizard (development)
   - DevOps pipeline with AzureWebApp@1 task (production)
   - GitHub Actions azure/webapps-deploy (open source projects)

Q15. Options to deploy Web API on Azure.

App Service: simplest option. PaaS, managed, auto-scale, slots, no Docker needed.
Use when: standard web API, steady traffic, team not familiar with containers.

AKS (Kubernetes): containerised microservices at scale.
Use when: multiple services, need fine-grained scaling per service.

Azure Container Apps: managed containers without full K8s complexity.
Use when: want containers but not K8s management overhead.

Azure Functions: serverless, event-driven.
Use when: API has very irregular traffic, want scale-to-zero.

Azure VM: full OS control.
Use when: legacy software, custom OS settings, lift-and-shift.

Q16. Why choose App Service to deploy WebAPI?

Managed infrastructure: no OS patching, no runtime installation.
Auto-scaling: scale out to multiple instances automatically on traffic.
Deployment slots: zero-downtime deploys with instant rollback.
Easy CI/CD: native integration with GitHub Actions and Azure DevOps.
Built-in monitoring: Application Insights integration with one click.
SSL management: free managed certificates, automatic renewal.

Custom domains: point your domain, SSL provisioned automatically.
Affordable: B1 tier (~$13/month) for small APIs.

Q17. Azure App Service vs Azure VM.

App Service (PaaS):
  Azure manages: OS patches, .NET runtime, IIS/Kestrel, load balancer.
  You manage: app code and configuration only.
  Deployment: push code/ZIP — done.
  Scaling: one slider in portal.
  Cost: pay for the tier (predictable).

VM (IaaS):
  You manage: OS patches, antivirus, .NET runtime installation, IIS config, firewall.
  Deployment: you RDP in and copy files, or set up your own CI/CD.
  Scaling: create more VMs, configure load balancer manually.
  Cost: VM + OS licence + disk + networking (adds up).
  Use for: software that cannot run on PaaS (old COM+ components, 32-bit only, etc.)

Q18. Azure Functions used with .NET.

HTTP Trigger: Function becomes a REST API endpoint.
  [HttpTrigger(AuthorizationLevel.Function, "get", "post")] HttpRequestData req
  Returns HttpResponseData. Same as a controller action.

Timer Trigger: scheduled background job (cron syntax).
  [TimerTrigger("0 0 8 * * *")] = run every day at 8:00 AM UTC
  Good for: nightly reports, data sync, cleanup jobs.

Service Bus Trigger: process messages from queue/topic automatically.
  [ServiceBusTrigger("orders-queue")] string messageBody
  Each message = one function invocation. Auto-scales with queue depth.

Blob Trigger: fires when file uploaded to container.
  [BlobTrigger("uploads/{name}")] Stream blobStream
  Good for: image processing, CSV parsing, virus scanning on upload.

Q19. How to secure Azure Function.

Function Keys (built-in): caller must include ?code=yourfunctionkey in URL.
  Three levels: anonymous (public), function key (per-function), admin key (all functions).

Azure AD (enterprise):
  Enable Authentication (EasyAuth) on Function App in portal.
  Azure AD validates JWT before request reaches your code.

Managed Identity for outbound calls:
  Function uses its own identity to call Key Vault, Storage, Service Bus.
  No credentials stored anywhere.

APIM in front:
  Azure API Management validates JWT, rate-limits, IP-filters before reaching Function.
  Function key not exposed to callers — APIM uses it internally.

VNet Integration:
  Function accessible only from inside private VNet.
  No public endpoint at all.

Q20. Storage account private but Function reads data — how?

Goal: Storage account has no public access, but Function needs to read blobs.

Step 1: Enable System Managed Identity on the Function App.
  Portal: Function App → Identity → System assigned → Status: On → Save.
  Azure creates an identity for your Function in Azure AD.

Step 2: Grant permission.
  Storage Account → Access Control (IAM) → Add role assignment.
  Role: Storage Blob Data Reader.
  Member: your Function App's managed identity.

Step 3: Use DefaultAzureCredential in code.

```
var client = new BlobServiceClient(
    new Uri("https://myaccount.blob.core.windows.net/"),
    new DefaultAzureCredential());  // uses managed identity automatically in Azure
```

No connection string, no storage key, no secret anywhere.
Azure identity system handles authentication invisibly.

Q21. What are durable function apps?

Extension for long-running, stateful workflows in Azure Functions.
Normal functions are stateless and short-lived (timeout ~5-10 min).
Durable functions can run for days, weeks, or months.

Two function types:
  Orchestrator: coordinates the workflow. Can call activity functions,
    wait for results, fan-out/fan-in, wait for external events.
  Activity: does the actual work (one unit of work per activity).

Common patterns:

Function Chaining:
  Orchestrator calls A → waits → calls B with A result → waits → calls C.
  Example: validate order → reserve inventory → charge payment → send email.

Fan-out/Fan-in:
  Orchestrator starts 100 parallel activity functions.
  Waits for all 100 to complete.
  Aggregates results.
  Example: process 100 files in parallel, then merge results.

Async HTTP:
  Client calls API → 202 Accepted + status URL returned immediately.
  Client polls status URL while long job runs.
  When done: status URL returns 200 with result.

Q22. Check function app is healthy?

Add a dedicated Health Check HTTP Function:
[Function("Health")]
[HttpTrigger(AuthorizationLevel.Anonymous, "get", Route = "health")]
Returns: 200 OK with { status: "healthy", timestamp: "..." }

Application Insights Availability Test:
App Insights → Availability → Add test.
Ping your /health URL every 5 minutes from 5 global regions.
If any ping fails: alert fires.
Detects regional outages automatically.

Azure Monitor Alert:
Alert rule on Function execution count dropping to zero (app not running).
Alert on exception count > 0 in 5-minute window.

Q23. Receive email if function fails — how to configure.

Steps in Azure Portal:

1. App Insights → Alerts → Create alert rule.
2. Scope: select your Function App.
3. Condition: Exceptions → Aggregation: Count → Operator: Greater than → Threshold: 0.
4. Evaluation: every 5 minutes, looking back 5 minutes.
5. Action Group: create new → Add action: Email/SMS/Push.
   Enter your email address.
6. Alert rule name: "Function Exception Alert".
7. Save.

Now: any unhandled exception in your Function → App Insights detects within 5 min → Alert fires → Email sent to you automatically.

Q24. Azure Key Vault — why important.

Key Vault solves the "secret in code" problem.

Without Key Vault (bad):
Connection string in appsettings.json → committed to git → visible to everyone.
Developer leaves company → secret is compromised → must rotate everything.

With Key Vault (good):
Secret stored in Key Vault (encrypted at rest with Azure-managed keys).
App accesses via Managed Identity — no credentials to retrieve credentials.
Access log: see exactly who/what accessed which secret and when.
Rotation: update secret in Key Vault — all apps pick up new value automatically.
Certificate management: auto-renew TLS certificates.

Q25. Managed Identity in Azure.

Azure AD identity automatically created and managed by Azure for a resource.
No passwords, no client secrets, no rotation needed.

Two types:

System Managed Identity:
  Created for one specific resource (e.g., one App Service).
  Lifecycle tied to the resource — deleted when resource is deleted.
  Cannot be shared with other resources.
  Enable: App Service → Identity → System assigned → On.

User Managed Identity:
  Standalone Azure resource you create and manage.
  Assign to multiple resources (e.g., 5 App Services share one identity).
  Persists even if resources are deleted.
  Use when: multiple resources need same permissions, or you pre-create identity for IaC.

Q26. Key Vault — how to connect from .NET?

Step 1: Enable Managed Identity on App Service or Function App (portal toggle).
Step 2: Grant access in Key Vault.
  Key Vault → Access Control (IAM) → Add role assignment.
  Role: Key Vault Secrets User (read only) or Key Vault Secrets Officer (read+write).
  Member: your App Service / Function App managed identity.
Step 3: Add NuGet packages.
  Azure.Identity, Azure.Security.KeyVault.Secrets
Step 4: Use in code — DefaultAzureCredential handles all auth automatically.

Q27. CI/CD for .NET project in Azure.

azure-pipelines.yml file lives in repo root alongside code.
Changes to pipeline are reviewed in PR like any other code change.

Pipeline stages:
  1. Build stage: restore → build → test → coverage → publish artifact.
  2. Dev stage: deploy artifact to Dev App Service automatically.
  3. Staging stage: deploy to Staging (can be automatic or manual trigger).
  4. Production stage: manual approval required → deploy → swap slots.

Environment-specific config:
  Connection strings / API keys stored in Azure DevOps Variable Groups.
  Referenced in YAML as $(DbConnectionString) — never hardcoded.

Q28. Azure Service Bus — how it works with .NET.

Architecture:
  Producer publishes message → Service Bus Queue or Topic.
  Consumer reads message → processes → acknowledges.

Queue: point-to-point. One message → one consumer processes it.
Topic + Subscriptions: pub/sub. One message → each subscription gets a copy.

Message lifecycle:
  1. Message arrives in queue.
  2. Consumer receives → message locked (not visible to others for lock duration).
  3. Consumer processes.
  4. Success: CompleteMessageAsync → message deleted from queue.
  5. Failure: AbandonMessageAsync → message unlocked, retried.
  6. After max retries (default 10): moved to Dead-Letter Queue automatically.

Q29. Service Bus — queue or topic?

Queue: ONE producer → ONE consumer processes each message.
Example: Customer places order → order goes to queue → warehouse service picks up.
Each order processed by exactly one warehouse worker.
Use when: work needs to be done once by one consumer.

Topic + Subscriptions: ONE producer → MULTIPLE consumers each get a copy.
Example: Order placed → order event published to topic.
Subscription 1 (shipping): get all orders to arrange shipping.
Subscription 2 (billing): get all orders to generate invoice.
Subscription 3 (analytics): get all orders to update dashboard.
Each subscription processes independently — no interference.

Q30. Event Grid — when and why?

Event Grid = Azure native event routing service.

Purpose: react to things that happen in Azure and route to handlers.

Event sources (built-in):
Blob Storage: file uploaded, file deleted.
Resource Manager: VM created, resource deleted.
Service Bus: message dead-lettered.
Container Registry: image pushed.
Custom: your app publishes custom events.

Handlers:
Azure Function, Logic App, Event Hub, Webhook, Service Bus.

Why use it:
Automation without polling. Instead of checking every 5 min "was a file uploaded?",
Event Grid calls your Function within milliseconds of the upload.
Loose coupling: uploader does not know about processor.

Q31. Event Hub — explain with example.

Event Hub = managed high-throughput event streaming (like Apache Kafka).

Capacity: millions of events per second.
Partitioned: data split across partitions (up to 32) for parallel processing.
Consumer Groups: multiple independent readers of same stream.
Retention: data kept for configurable period (1-7 days) — replay possible.

Real example (Qatar Airways use case):
Aircraft: 1000 sensors send GPS, speed, fuel, engine data every second.
That is 1,000,000 events/second during peak.

Event Hub receives all events.
Consumer Group 1 (real-time dashboard): reads stream, updates live map.
Consumer Group 2 (anomaly detection): ML model flags engine issues.
Consumer Group 3 (cold storage): writes to Azure Data Lake for later analysis.

All three consumers process the SAME events independently.
None blocks the others.

Q32. What is Data mart?

A Data Mart is a focused subset of a data warehouse for one business area.

Example:
Enterprise Data Warehouse: all data (Sales, Finance, HR, Logistics, Marketing).
Sales Data Mart: only Sales data, optimised for Sales KPI queries.
Finance Data Mart: only Finance data, optimised for financial reports.

Benefits:
Faster queries: smaller dataset, indexed for department-specific queries.
Simpler: analysts do not need to understand the full warehouse schema.
Access control: Finance team only sees Finance mart.

Where it fits: Operational DB → ETL → Data Warehouse → Data Mart → BI Tool (Power BI).

Q33. Logging and Monitoring in Azure for .NET.

Setup (Program.cs):
builder.Services.AddApplicationInsightsTelemetry();

What App Insights auto-captures:
All HTTP requests: URL, status code, duration, client IP.
All DB queries: SQL text, duration, success/failure.
All outgoing HTTP calls: URL, status, duration.
All unhandled exceptions: full stack trace, request context.

Custom tracking:
_logger.LogInformation("Order {Id} created by {User}", orderId, userId);
_telemetryClient.TrackEvent("PaymentProcessed", new { amount = 100 });

View in Azure Portal:
Failures tab: all exceptions grouped by type.
Performance tab: slowest requests with drill-down to DB calls.
Live Metrics: real-time request rate, failure rate, response time.

Q34. Azure APIM (API Management).

Single front door for all your APIs regardless of where they are hosted.

What APIM does:
ROUTING: /api/orders → App Service 1, /api/products → App Service 2.
AUTHENTICATION: validate-jwt policy checks token before reaching your API.
RATE LIMITING: 100 requests/minute per subscription key.
IP FILTERING: allow only specific IP ranges.
TRANSFORMATION: add/remove headers, change JSON structure.
MOCKING: return fake response during development.
DOCUMENTATION: developer portal generated from OpenAPI spec.
VERSIONING: /v1/ and /v2/ routes to different backends.
ANALYTICS: dashboard showing API usage, top consumers, error rates.

Your API code stays clean — cross-cutting concerns are APIM policies.

Q35. Secure ASP.NET Core API in Azure.

Layer 1 — Edge: Azure Front Door with WAF (Web Application Firewall).
Blocks DDOS, SQL injection, XSS, bot traffic before it reaches your API.

Layer 2 — Gateway: Azure APIM.
Validates JWT. Rate limits. IP filtering. Hides backend URL from clients.

Layer 3 — App Service: HTTPS only enforced.
HTTP → automatic redirect to HTTPS. No unencrypted traffic allowed.

Layer 4 — Application code:
[Authorize] attribute on controllers.
Role/policy-based authorisation.
Input validation (FluentValidation).
No secrets in code (Key Vault).

Layer 5 — Data: Private endpoints.
SQL Server and Key Vault accessible only from App Service VNet subnet.
Not reachable from public internet at all.

Q36. AI services in Azure?

Azure OpenAI Service:
GPT-4, GPT-3.5 Turbo, DALL-E 3, text embeddings.
Same models as OpenAI but hosted in Azure with enterprise security/compliance.
Your data does not train the model.

Azure AI Search (formerly Cognitive Search):
Vector search + semantic ranking.
Key for RAG: store document embeddings, search by similarity.

Azure Document Intelligence (Form Recognizer):
Extract structured data from PDFs, receipts, invoices.
Pre-built models or custom trained on your documents.

Azure AI Vision:
Image analysis, OCR, face detection, object detection.

Azure ML:
Train custom ML models. Auto ML. MLOps pipelines. Model registry.

Q37. Have you used AKS?

AKS = Azure Kubernetes Service — managed Kubernetes.
Azure manages control plane (API server, scheduler, etcd).
You manage worker nodes and workloads.

How microservices deploy:
Each service: Docker image → pushed to ACR → Helm chart defines Deployment + Service.
Deployment YAML: replicas, container image, resource limits, environment vars.
Service YAML: exposes pods inside cluster (ClusterIP for internal).

Ingress resource: nginx ingress controller routes external HTTP to services.

Scaling:
HPA (HorizontalPodAutoscaler): scales pods on CPU/custom metrics.
Cluster Autoscaler: adds/removes VM nodes based on pending pods.

Managed Identity integration:
Workload Identity: pod gets Azure AD identity → calls Key Vault, Service Bus, Storage.
No secrets in pod manifests.

Q38. App Insights — single place for all apps?

Yes — with Workspace-based Application Insights (modern approach).

Setup:
1. Create one Log Analytics Workspace (shared).
2. For each App Insights resource: Properties → Workspace → select shared workspace.
All telemetry flows to same workspace.

Query across all apps in one KQL query:

```
union app("api-service").requests,
      app("auth-service").requests,
      app("worker-service").requests
| where timestamp > ago(1h)
| summarize count() by appName, resultCode
```

One dashboard shows health of entire system.
One alert rule can trigger on error from any service.

Q39. Configure deployment parameters — pods in K8s.

Deployment YAML (per microservice):
spec.replicas: 3 — desired number of pods.
spec.strategy.rollingUpdate.maxUnavailable: 1 — allow 1 pod down during update.
spec.template.spec.containers[0].resources:
requests.cpu: "250m" — minimum CPU to schedule pod (1000m = 1 core).
limits.cpu: "500m" — hard maximum CPU.
requests.memory: "256Mi"
limits.memory: "512Mi"

HorizontalPodAutoscaler (separate YAML):
minReplicas: 2 — always at least 2 pods running.
maxReplicas: 20 — never exceed 20.
targetCPUUtilizationPercentage: 70 — scale up if average CPU > 70%.

Helm values.yaml: environment-specific values override template defaults.
values-dev.yaml: replicas: 1, limits.cpu: 250m (save cost in dev).
values-prod.yaml: replicas: 3, limits.cpu: 500m (full performance in prod).

Q40. Azure Storage Account in .NET.

Install NuGet: Azure.Storage.Blobs

Connect using Managed Identity (no connection string needed):

```
var blobSvc = new BlobServiceClient(
    new Uri("https://myaccount.blob.core.windows.net/"),
    new DefaultAzureCredential());
```

Get container: `var container = blobSvc.GetBlobContainerClient("documents");`

Upload file:

```
using var stream = File.OpenRead("report.pdf");
await container.GetBlobClient("2024/q1-report.pdf").UploadAsync(stream, overwrite:true);
```

Download file:

```
var blob = container.GetBlobClient("2024/q1-report.pdf");
var response = await blob.DownloadStreamingAsync();
```

List all blobs:

```
await foreach (var item in container.GetBlobsAsync())
    Console.WriteLine(item.Name);
```

Q41. How to use Azure Key Vault?

Two approaches:

A) Configuration Provider — auto-loads all secrets into IConfiguration.
Easiest. Works like appsettings.json.
Add NuGet: Azure.Extensions.AspNetCore.Configuration.Secrets

```
builder.Configuration.AddAzureKeyVault(
    new Uri("https://myvault.vault.azure.net/"),
    new DefaultAzureCredential());
```

Usage: `string connStr = _config["DbConnectionString"];`

B) SecretClient — direct programmatic access.
Add NuGet: Azure.Security.KeyVault.Secrets

```
var client = new SecretClient(
    new Uri("https://myvault.vault.azure.net/"),
    new DefaultAzureCredential());
var secret = await client.GetSecretAsync("DbPassword");
string password = secret.Value.Value;
```

Q42. How to fetch values from Key Vault?

After AddAzureKeyVault (Option A):
All secrets automatically available via IConfiguration.
`string dbConn = _configuration["DatabaseConnectionString"];`
Same API as reading from appsettings.json — no Key Vault code in controllers.

SecretClient (Option B):

```
var result = await _secretClient.GetSecretAsync("DatabaseConnectionString");
string value = result.Value.Value;
```

Note: result.Value is KeyVaultSecret, .Value is the actual secret string.

Caching tip:
Do NOT call GetSecretAsync on every HTTP request.
Cache the value: `private string _cachedConn;`
Load once in service constructor or via IOptions<T>.

Key Vault has request limits and calling it per-request is wasteful.

Q43. Web API vs Azure Functions — why both?

They solve different problems.

Web API (App Service):
- Always running, always warm, no cold start.
- Complex routing (/api/v2/orders/{id}/items).
- Full middleware pipeline (auth, CORS, rate limiting, caching).
- DI container with scoped lifetimes.
- Use for: user-facing APIs, consistent traffic, complex business logic.

Azure Functions:
- Scale to zero — no traffic = no cost.
- Event-driven: triggered by queue, timer, blob, webhook.
- Cold start acceptable for background work.
- Use for: nightly reports, queue processing, file processing, webhooks.

Typical architecture:
- App Service hosts the REST API (frontend talks to this).
- Azure Functions handle background work (email sending, data sync, file processing).
- Together they cover all scenarios at optimal cost.

Q44. appsettings vs Key Vault — when?

appsettings.json — safe for source control, non-sensitive:
- Log level configuration.
- Feature flags (isMaintenanceModeEnabled: false).
- Timeout values (requestTimeoutSeconds: 30).
- Public API endpoint URLs (externalApiBaseUrl: "https://api.example.com").
- Pagination defaults (defaultPageSize: 20).

Azure Key Vault — never in source control, sensitive:
- Database connection strings (contain password).
- OAuth client secrets (used for token generation).
- Third-party API keys (payment gateway, SMS provider).
- Encryption keys.
- TLS/SSL certificate private keys.
- Internal service passwords.

Rule: if leaked on GitHub, would it be a problem? → Key Vault.

Q45. How to access Key Vault in different environments.

In Azure (App Service or Function App):
- DefaultAzureCredential auto-detects Managed Identity.
- Zero configuration needed. Just enable managed identity in portal.

Local development:
- DefaultAzureCredential falls through chain:
- Tries environment variables (AZURE_CLIENT_ID etc.) → az login → Visual Studio login.
- Run: az login → developer can access Key Vault with their own Azure AD identity.

CI/CD pipeline (GitHub Actions / Azure DevOps):
Create Service Principal: az ad sp create-for-rbac.
Store credentials in GitHub Secrets or DevOps Variable Group.
Set AZURE_CLIENT_ID, AZURE_TENANT_ID, AZURE_CLIENT_SECRET env vars.
DefaultAzureCredential picks up env vars automatically.

Q46. Function apps vs App Services.

Choose based on traffic pattern and requirements.

Azure Functions (choose when):
Traffic is unpredictable / bursty.
Events drive processing (queue messages, file uploads, timers).
Want scale-to-zero to save cost (dev/test environments especially).
Short-lived processing (< 10 minutes per invocation).
Pay-per-execution billing preferred.

App Service (choose when):
Consistent, steady traffic where always-on is needed.
Complex routing, versioning, DI with multiple lifetimes.
Long-running processes (WebSockets, SSE streaming).
Need deployment slots for zero-downtime.
Simpler operational model for the team.

Q47. Authentication Approach — Azure AD.

Flow:
1. Register application in Azure AD (Entra ID).
Gets: Application (client) ID, Tenant ID.
Set redirect URI, expose API scope.
2. Client application requests token.
POST to: https://login.microsoftonline.com/{tenantId}/oauth2/v2.0/token
With: client_id, client_secret (or PKCE for SPA), scope, grant_type.
3. Client sends token in request header.
Authorization: Bearer eyJhbGci...
4. API validates token automatically via middleware.
builder.Services.AddMicrosoftIdentityWebApi(configuration);
Validates: signature, issuer, audience, expiry.
5. Claims available in code.
var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;

Q48. What is JWT.

JSON Web Token — self-contained token for stateless authentication.

Structure: three Base64URL-encoded parts separated by dots.
Part 1 — Header: algorithm and token type.
{ "alg": "HS256", "typ": "JWT" }
Part 2 — Payload: claims (facts about the user).
{ "sub": "user42", "name": "Vamshi", "role": "Admin", "exp": 1700000000 }
Part 3 — Signature: proves token was not tampered with.
HMACSHA256(Base64(header) + "." + Base64(payload), secretKey)

Validation:

Server recomputes signature using its secret key.
If computed signature matches token signature → token is genuine.
Also checks: exp (not expired), iss (correct issuer), aud (correct audience).

Stateless: server stores NOTHING. All user info in token. Scales horizontally.

Q49. OAuth 2.0 vs OpenID Connect.

OAuth 2.0 = AUTHORIZATION framework.
Purpose: "Allow this app to access my files on Google Drive."
Grants: access_token (opaque or JWT).
Answers: what can this application DO on behalf of the user?
Does NOT tell you who the user is.

OpenID Connect (OIDC) = AUTHENTICATION layer on top of OAuth 2.0.
Purpose: "Sign in with Google."
Grants: access_token PLUS id_token (JWT with user identity claims).
Answers: WHO is this user? (name, email, photo, verified claims).
id_token contains: sub (unique user ID), name, email, picture.

When to use:
User login / identity → OIDC.
API access / delegated permissions → OAuth 2.0.
In practice: most identity providers (Azure AD, Google) support both together.

Q50. Client Credential Flow.

Used when: no human user is involved. Machine-to-machine communication.

Example scenario:
Notification service (background job) needs to call Orders API.
No user is logged in — it is an automated process.

Flow:
1. Notification service sends POST to token endpoint.
   grant_type=client_credentials, client_id=abc, client_secret=xyz, scope=api://orders/.default
2. Azure AD returns access_token (JWT).
3. Notification service calls Orders API with: Authorization: Bearer <token>
4. Orders API validates token — sees it came from Notification service app registration.
5. Orders API allows or denies based on app roles.

Common uses: scheduled jobs, microservice-to-microservice, CI/CD pipeline automation.

Q51. Types of Identities in Azure.

System Managed Identity:
Azure creates automatically when you enable it on a resource (App Service, Function, VM).
Exists only for that one resource.
Deleted automatically when the resource is deleted.
Good for: simple single-resource scenarios.

User Managed Identity:
You create it as a standalone Azure resource.
Assign to any number of resources (5 App Services share one identity).

Persists independently — deleting an App Service does not delete the identity.
Good for: multiple resources sharing same permissions, IaC where identity created before resource.

User Identity / Service Principal:
Actual Azure AD account (human user or application with client ID + secret).
Used in: CI/CD pipelines (service principal), local development access.

Q52. .NET Web API consuming SQL + Blob — Azure integration.

Architecture: App Service → SQL Server (private) + Blob Storage (private) + Key Vault.

Enable System Managed Identity on App Service (portal toggle).

SQL connection — passwordless:
Connection string: "Server=myserver.database.windows.net;Database=MyDb;Authentication=Active Directory Managed Identity"
No password in connection string. Azure AD identity used to authenticate to SQL.
SQL Server side: CREATE USER [MyAppServiceName] FROM EXTERNAL PROVIDER; GRANT SELECT, INSERT ON SCHEMA::dbo TO [MyAppServiceName];

Blob — passwordless:
var blobSvc = new BlobServiceClient(new Uri(blobEndpoint), new DefaultAzureCredential());
Grant role: Storage Blob Data Contributor on storage account to managed identity.

Key Vault — passwordless:
builder.Configuration.AddAzureKeyVault(vaultUri, new DefaultAzureCredential());
Grant role: Key Vault Secrets User on Key Vault to managed identity.

Result: zero secrets anywhere.

Q53. Background recurring job — daily notifications.

Option 1: Azure Functions Timer Trigger (RECOMMENDED for new projects).
[TimerTrigger("0 0 8 * * *")] — fires every day at 8:00 AM UTC.
Cron format: seconds minutes hours day month weekday.
Scales automatically. No server to manage.

Option 2: Hangfire (if using App Service).
services.AddHangfire(...); services.AddHangfireServer();
RecurringJob.AddOrUpdate("daily-notify", () => svc.SendNotifications(), Cron.Daily);
Dashboard at /hangfire shows job history and failures.

Option 3: Azure Logic Apps.
No-code scheduler. Drag-and-drop triggers and actions.
Good for non-developers or complex approval workflows.

Option 4: Azure Container Apps Job.
Run containerised job on schedule.
Good for complex jobs that need custom runtime environment.

Q54. WebJobs.

Background tasks that run inside an Azure App Service context.

Types:
  Continuous WebJob: runs permanently alongside web app (never stops).
    Good for: queue listener, always-on background processor.
  Triggered WebJob: runs on demand via URL or on schedule via cron.
    Good for: nightly cleanup, scheduled reports.

Access: Azure Portal → App Service → WebJobs tab.
Logs: each WebJob run has its own log viewable in portal.

Limitation: shares resources with the web app (CPU, memory).
Modern alternative: Azure Functions is preferred for new development.

Q55. 2 WebJobs in same App Service Plan?

Yes — multiple WebJobs run under same App Service Plan.
All WebJobs share: CPU, RAM, disk of the plan.

Problem: if WebJob A is CPU-intensive, it starves WebJob B and the web app.

Solution options:
  Scale UP the plan (bigger VM — more CPU/RAM for all).
  Move resource-intensive WebJobs to a separate App Service Plan.
  Migrate WebJobs to Azure Functions with dedicated plan (no resource sharing).

Q56. Decide separate App Service Plan.

Same plan: apps/jobs that have complementary resource usage.
  Web app: busy during business hours. Batch job: runs only at night. Share plan safely.

Separate plans when:
  Different scaling: API needs 10 instances at peak, worker needs 1.
    Same plan would scale both to 10 — wasteful.
  Performance isolation: CPU-heavy job causes latency on web API → separate.
  Different OS/stack: .NET API on Linux, legacy job requires Windows.
  Cost accountability: different teams billed separately.
  Availability: if plan is full, both services are affected — separate for independence.

Q57. Track issues in Functions / App Services.

Application Insights (primary tool):
  Add SDK: builder.Services.AddApplicationInsightsTelemetry();
  Auto-captures: every exception with full stack trace, every request with timing.

Drill-down workflow:
  Failures tab → click exception type → see all occurrences.
  Click one occurrence → full stack trace + custom properties you logged.
  Click Transaction ID → see everything that happened in that request:
    DB queries executed (with SQL text and duration).
    External HTTP calls made.
    Custom events and traces.
    Other exceptions in same request.

Alert setup: when exceptions/count > 0 → email notification.

Q58. Azure Monitor vs Application Insights.

Application Insights:
Application Performance Monitoring (APM).
Tracks your APPLICATION behaviour.
Data: HTTP request rates, response times, exception details, custom events, traces.
Who uses it: developers, SREs diagnosing app bugs.

Azure Monitor:
Infrastructure and platform monitoring.
Tracks Azure RESOURCES.
Data: VM CPU %, disk IOPS, network throughput, SQL DTU usage, AKS node health.
Who uses it: infrastructure team, operations.

Relationship:
Azure Monitor is the umbrella. App Insights feeds its data INTO Azure Monitor.
Both stored in Log Analytics Workspace.
Can query both together in KQL.

Q59. Publisher/Subscriber pattern — track and endpoint.

Setup with Azure Service Bus Topic:
1. Create Topic (e.g., "order-events").
2. Create Subscription per consumer (e.g., "shipping", "billing", "analytics").
3. Each consumer connects to its own subscription — independent queue.

Producer sends once → all 3 subscriptions each get a copy.

Monitoring:
Azure Portal → Service Bus → Metrics tab.
Active Message Count: how many messages waiting to be processed.
Dead-letter Message Count: failed messages needing investigation.
Message Rate: throughput (messages per second).

App Insights on consumer side tracks processing duration and failures.
Correlation ID in message properties links publisher trace to consumer trace.

Q60. Service Bus DeadLetter Queue — code level.

Message lifecycle in code:

SUCCESS: processing complete, message removed from queue.
```
await args.CompleteMessageAsync(args.Message);
```

TRANSIENT FAILURE: temporary error (DB timeout, network), should retry.
```
await args.AbandonMessageAsync(args.Message);
```
Message goes back to queue. Lock expires. Other consumers can pick it up.
After MaxDeliveryCount (default 10) failed attempts → auto moved to dead-letter.

PERMANENT FAILURE: data invalid, business rule violated, no point retrying.
```
await args.DeadLetterMessageAsync(args.Message,
    deadLetterReason: "ValidationFailed",
    deadLetterErrorDescription: "Required field OrderId is missing");
```
Message moved to $DeadLetterQueue immediately.
Monitor dead-letter count. Investigate and fix, then replay or discard.

Q61. Single Topic, multiple subscribers, filter by field.

SQL Filter on Subscription — filter messages by application property.

When sending (producer sets application property):

```
message.ApplicationProperties["OrderType"] = "Premium";
message.ApplicationProperties["Region"]    = "APAC";
```

Create subscription with filter:

```
var rule = new CreateRuleOptions {
    Name = "PremiumOnly",
    Filter = new SqlRuleFilter("OrderType = 'Premium'")
};
await adminClient.CreateSubscriptionAsync(topicName, subscriptionName, rule);
```

Result:

"Premium" subscription: only receives OrderType=Premium messages.
"Standard" subscription: filter "OrderType = 'Standard'".
"APAC" subscription: filter "Region = 'APAC'".
Each subscription independently filters the same topic stream.

Q62. Service Bus trigger ways in .NET.

1. Azure Function ServiceBusTrigger (RECOMMENDED for serverless):

```
[ServiceBusTrigger("orders-queue")] string messageBody
```

Automatically scales with queue depth. Each message = one invocation.

2. ServiceBusProcessor SDK (for App Service BackgroundService):

```
var processor = client.CreateProcessor("orders-queue");
processor.ProcessMessageAsync += HandleMessage;
await processor.StartProcessingAsync();
```

3. Logic Apps connector: no-code trigger, good for workflow automation.
4. Event Grid bridge: Service Bus dead-letter event → trigger a Function.
5. Service Bus REST API: any HTTP client (mobile, external service) can receive.

Q63. Access Key Vault from .NET — class level.

Key classes and their roles:

SecretClient: manages secrets (most common — connection strings, API keys).
CertificateClient: manages TLS certificates.
KeyClient: manages cryptographic keys for encryption operations.

Pattern used in production:

Register as singleton (Key Vault client is thread-safe, reuse is correct).
Call GetSecretAsync during app startup or first request.
Cache the value in a private field.
Do not call Key Vault on every request (rate limited + latency).

Dependency injection:

```
builder.Services.AddSingleton(sp =>
    new SecretClient(new Uri(vaultUri), new DefaultAzureCredential()));
```

Then inject into constructor: SecretClient _secretClient.

Q64. AKS deployment — Helm, Ingress, CI/CD.

Helm Chart structure:
/myservice/
Chart.yaml — name, version, description
values.yaml — default config (overridden per environment)
templates/
deployment.yaml — K8s Deployment (pods)
service.yaml — K8s Service (stable DNS)
ingress.yaml — HTTP routing rules

values.yaml (production):
image.repository: myacr.azurecr.io/myservice
image.tag: latest (overridden by CI with build number)
replicaCount: 3
ingress.host: api.mycompany.com

CI pipeline: build image → docker push myacr.azurecr.io/myservice:BuildId
CD pipeline: helm upgrade --install myservice ./helm/myservice
--set image.tag=BuildId
-f values-prod.yaml

Q65. ACR — what is it?

Azure Container Registry = private Docker image registry hosted in Azure.

Why not Docker Hub?
Docker Hub: public by default, rate limited, private repos cost money.
ACR: private, integrated with Azure RBAC, no rate limits, geo-replication.

Integration with AKS:
az aks update --attach-acr myacr (grants AKS managed identity pull access).
AKS pulls images from ACR without credentials in pod manifests.

Key features:
Image vulnerability scanning (Microsoft Defender for Containers).
Geo-replication: same image available in multiple regions.
Tasks: build images in Azure on git push (like a mini CI pipeline).
Retention policy: auto-delete old untagged images.

Q66. Container vs Pod.

Container:
A running instance of a Docker image.
One isolated process with its own filesystem and network.
Managed by Docker runtime.

Pod:
Kubernetes smallest deployable unit.
Wraps one or more containers that must run together on the same node.
All containers in pod share:
Network namespace (talk to each other via localhost, same IP).
Storage volumes (mounted at same paths).

Lifecycle (start and stop together).

Common pod patterns:
Single container: 99% of cases (one app per pod).
Sidecar: main app + log collector, or main app + service mesh proxy (Envoy).
Init container: runs to completion before main container starts (DB migration).

Q67. Helm Charts — what do they do, why needed.

Without Helm:
Write 6 YAML files per service (Deployment, Service, Ingress, ConfigMap, HPA, PDB).
Duplicate them for each environment with manual value changes.
Deploy: kubectl apply -f file1.yaml -f file2.yaml ... (error-prone).
Rollback: manually find old YAML and reapply.

With Helm:
Templates with {{ .Values.replicaCount }} placeholders.
One values.yaml per environment (values-dev.yaml, values-prod.yaml).
Deploy: helm install myapp ./chart -f values-prod.yaml (one command).
Rollback: helm rollback myapp 1 (go back to revision 1).
History: helm history myapp (see all deployments with timestamps).
Package: helm package ./chart → myapp-1.0.0.tgz for distribution.

Q68. CLI to test Helm Chart locally.

Render templates without deploying (inspect generated YAML):
helm template myapp ./chart -f values-dev.yaml
Output: all K8s YAML that would be applied. Check for errors.

Dry run (simulate install with K8s API validation):
helm install myapp ./chart --dry-run --debug -f values-dev.yaml
K8s validates YAML structure without creating any resources.

Local K8s cluster:
kind create cluster (Kubernetes in Docker — free, fast, disposable).
helm install myapp ./chart -f values-dev.yaml
kubectl get pods → see real pods running locally.
Delete when done: kind delete cluster.

Q69. AKS — did you deploy yourself or DevOps team?

Typical enterprise answer:
DevOps/Platform team: provisions AKS cluster using Terraform.
Configures node pools, VNet integration, managed identity, RBAC.
Sets up nginx ingress controller cluster-wide.
Connects ACR to AKS.

Application team (developer): provides Helm chart and values files.
templates/deployment.yaml, service.yaml, ingress.yaml.
values-dev.yaml, values-prod.yaml.
Application team does not touch cluster config.

CI/CD pipeline (joint):
Build: developer-owned (build image, push to ACR).

Deploy: helm upgrade command run by pipeline using service principal.
Rollback: automated (helm rollback) or manual by developer.

Q70. Experience on VNet or Networking concepts.

VNet (Virtual Network):
Private isolated network in Azure. Resources inside talk privately.
You define IP address space (e.g., 10.0.0.0/16).
Divided into Subnets (10.0.1.0/24 for AKS, 10.0.2.0/24 for App Service).

Private Endpoints:
Give Azure PaaS services (SQL, Key Vault, Storage) a private IP inside your VNet.
No public internet exposure. Traffic never leaves Azure network.

NSG (Network Security Group):
Firewall rules per subnet or per NIC.
Rule: Allow HTTPS from internet to App Service subnet.
Rule: Allow SQL port 1433 only from App Service subnet.
Rule: Deny everything else.

VNet Integration for App Service:
Lets App Service reach resources inside VNet (private SQL, private Key Vault).
Outbound traffic from App Service routes through VNet.

Private DNS Zone:
myserver.database.windows.net resolves to private IP inside VNet.
Public DNS resolves to public IP (which is disabled).
Resources use private name, private IP, private path.

Azure Code Examples — Key Vault + Service Bus + Blob

QAZ1. Complete Azure integration — Managed Identity + Key Vault + Service Bus + Blob

Program.cs + Service implementations:

```
// ── Program.cs — register all Azure services (no secrets anywhere) ──────────
var builder = WebApplication.CreateBuilder(args);

// Key Vault: load all secrets into IConfiguration automatically
// After this: _config["DbConnectionString"] works like appsettings.json
builder.Configuration.AddAzureKeyVault(
    new Uri("https://myvault.vault.azure.net/"),
    new DefaultAzureCredential());

// Blob Storage: Managed Identity auth (no storage account key)
builder.Services.AddSingleton(_ =>
    new BlobServiceClient(
        new Uri("https://myaccount.blob.core.windows.net/"),
        new DefaultAzureCredential()));

// Service Bus: Managed Identity auth
builder.Services.AddSingleton(_ =>
    new ServiceBusClient(
        "mybus.servicebus.windows.net",
        new DefaultAzureCredential()));

// ── Sending a message (OrderService.cs) ──────────────────────────────────────
public class OrderPublisher
{
    private readonly ServiceBusSender _sender;
```

```
    public OrderPublisher(ServiceBusClient client)
    {
        // One sender per queue/topic — create once, reuse (thread-safe)
        _sender = client.CreateSender("orders-topic");
    }

    public async Task PublishOrderCreated(Order order)
    {
        var payload = new
        {
            orderId    = order.Id,
            customerId = order.CustomerId,
            total      = order.Total,
            timestamp  = DateTime.UtcNow
        };

        var message = new ServiceBusMessage(BinaryData.FromObjectAsJson(payload));

        // Correlation ID links this message to the HTTP request trace in App Insights
        message.CorrelationId = Activity.Current?.Id;

        await _sender.SendMessageAsync(message);
        // Returns immediately — producer does not wait for consumer to process
    }
}

// ── Consuming messages (OrderConsumer.cs — runs as hosted background service) ─
public class OrderConsumer : BackgroundService
{
    private readonly ServiceBusProcessor _processor;
    private readonly ILogger<OrderConsumer> _log;

    public OrderConsumer(ServiceBusClient client, ILogger<OrderConsumer> log)
    {
        _log = log;
        _processor = client.CreateProcessor(
            topicName: "orders-topic",
            subscriptionName: "shipping-subscription",
            new ServiceBusProcessorOptions
            {
                MaxConcurrentCalls = 5,       // process 5 messages simultaneously
                AutoCompleteMessages = false   // we control complete/abandon manually
            });
    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        _processor.ProcessMessageAsync += OnMessage;
        _processor.ProcessErrorAsync   += OnError;
        await _processor.StartProcessingAsync(stoppingToken);
        // BackgroundService runs until app shuts down
    }

    private async Task OnMessage(ProcessMessageEventArgs args)
    {
        var order = args.Message.Body.ToObjectFromJson<OrderMessage>();
        _log.LogInformation("Processing order {Id}", order.OrderId);

        try
        {
            await ArrangeShipping(order);

            // SUCCESS: remove from queue permanently
            await args.CompleteMessageAsync(args.Message);
        }
        catch (TransientDbException)
        {
```

```
            // TEMPORARY: DB was momentarily down, put back in queue, retry shortly
            await args.AbandonMessageAsync(args.Message);
            // After 10 retries Service Bus auto-moves to dead-letter queue
        }
        catch (InvalidOrderException ex)
        {
            // PERMANENT: data is fundamentally wrong, retrying will never help
            await args.DeadLetterMessageAsync(args.Message,
                deadLetterReason: "InvalidOrder",
                deadLetterErrorDescription: ex.Message);
        }
    }

    private Task OnError(ProcessErrorEventArgs args)
    {
        _log.LogError(args.Exception, "Service Bus infrastructure error");
        return Task.CompletedTask;
    }
}

// ── Azure Function Timer Trigger (daily notification) ──────────────────────
public class NotificationFunction
{
    [Function("DailyNotification")]
    public async Task Run(
        [TimerTrigger("0 0 8 * * *")] TimerInfo timer,  // every day 8:00 AM UTC
        FunctionContext ctx)
    {
        var log = ctx.GetLogger<NotificationFunction>();
        log.LogInformation("Daily notification started at {Time}", DateTime.UtcNow);

        // timer.IsPastDue: true if function missed its schedule (app was down)
        if (timer.IsPastDue)
            log.LogWarning("Timer is past due — running catch-up execution");

        var pendingOrders = await GetPendingOrdersAsync();
        foreach (var order in pendingOrders)
            await SendReminderEmailAsync(order);

        log.LogInformation("Notified {Count} customers", pendingOrders.Count);
    }
}
```

💡 DefaultAzureCredential is the key. In Azure it uses Managed Identity automatically. Locally it uses your az login. One line of code works everywhere — no environment-specific authentication code.

DEVOPS & CI/CD

Complete Interview Q&A — Azure DevOps, CI/CD Pipelines, YAML, Agents

29 Questions · Blue-Green · Canary · Self-Hosted Agents · Release Pipelines · IIS Deployment

Section A: CI/CD Fundamentals (Q1–Q9)

Q1. Explain CI/CD pipeline

✅ **Answer:**

CI (Continuous Integration) = automatically build + test code on every commit. CD (Continuous Delivery/Deployment) = automatically deploy tested code to staging or production.

Stage	Action	Tool
Source	Code pushed to Git repo	Azure Repos / GitHub
CI — Build	Restore, build, run tests	dotnet build + dotnet test
CI — Test	Unit tests, code coverage	xUnit, coverlet
CD — Stage	Deploy to staging environment	Azure App Service / AKS
CD — Prod	Deploy to production (manual approval gate)	Release Pipeline + Approvals
Monitor	Track errors, performance	App Insights, Serilog

```
# azure-pipelines.yml — Full CI/CD example
trigger:
  branches:
    include: [main, develop]

pool:
  vmImage: 'ubuntu-latest'

variables:
  buildConfig: 'Release'
  dotnetVersion: '8.0.x'

stages:
- stage: CI
  jobs:
  - job: BuildAndTest
    steps:
    - task: UseDotNet@2
      inputs:
        version: $(dotnetVersion)
    - script: dotnet restore
    - script: dotnet build --configuration $(buildConfig)
    - script: dotnet test --collect:"XPlat Code Coverage"
    - task: PublishCodeCoverageResults@1

- stage: CD_Staging
  dependsOn: CI
  jobs:
  - deployment: DeployStaging
    environment: 'staging'
```

```
    strategy:
      runOnce:
        deploy:
          steps:
          - task: AzureRmWebAppDeployment@4
            inputs:
              azureSubscription: 'MyServiceConnection'
              appName: 'myapp-staging'
```

Q2. How do you perform deployment without downtime?

✅ **Answer:**

Use Blue-Green or Rolling deployment strategies so users always hit a live instance.

Strategy	How It Works	Downtime
Blue-Green	Two identical envs — route traffic to green after deploy	Zero — instant switch
Rolling	Replace instances one-by-one	Zero — old instances serve traffic during rollout
Canary	Route 5–10% traffic to new version first	Zero — gradual shift
Feature Flags	Deploy code off, turn on per user group	Zero — code ships dark

```
# Kubernetes Rolling Update — zero downtime default
apiVersion: apps/v1
kind: Deployment
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1        # allow 1 extra pod during update
      maxUnavailable: 0  # never take a pod down before new one is ready
```

Q3. Blue-Green vs Canary deployment

✅ **Answer:**

Feature	Blue-Green	Canary
Traffic split	All or nothing — 100% switch	Gradual — 5% → 25% → 100%
Rollback	Instant — switch back to blue	Gradual — reduce canary weight
Infrastructure	2× infra needed	Same infra, weighted routing
Risk	Higher — all users on new at once	Lower — limited blast radius
Use when	Fast rollback needed, confident release	Risky release, A/B test, feature validation

> 💡 *Azure Deployment Slots (App Service) make Blue-Green trivial — slot swap is atomic with zero downtime.*

Q4. How do you troubleshoot production issues?

✅ **Answer:**

Structured approach: Detect → Triage → Isolate → Fix → Verify → Post-mortem.

- 1. Check Azure Application Insights — exceptions, slow requests, dependency failures
- 2. Check Kusto logs / Log Analytics for error patterns
- 3. Check deployment timeline — did a recent release coincide with the issue?
- 4. Reproduce in staging using the same payload/conditions
- 5. Use feature flags to disable the problematic feature without redeployment
- 6. Hotfix branch → fast CI/CD pipeline → deploy fix → verify monitors
- 7. Write post-mortem: root cause, timeline, fix, prevention

Q5. Debugging strategies in live systems

✅ Answer:

- Structured logging with correlation IDs (Serilog + Seq/App Insights)
- Remote debugging via Azure App Service (attach VS debugger to live app)
- Application Insights Snapshot Debugger — captures call stack + locals on exception
- Health check endpoints — /health shows dependency status
- Feature flags — disable functionality without redeploy
- Read-only SQL traces — SQL Profiler or Extended Events on staging

```
// Correlation ID in Serilog for tracing across services
Log.ForContext("CorrelationId", correlationId)
   .ForContext("UserId", userId)
   .Error(ex, "Order processing failed for OrderId {OrderId}", orderId);

// App Insights custom event
_telemetry.TrackEvent("PaymentFailed", new Dictionary<string,string>{
    ["OrderId"]   = orderId.ToString(),
    ["Reason"]    = reason,
    ["UserId"]    = userId
});
```

Q6. How to implement CI/CD for a .NET project in Azure?

✅ Answer:

Full flow: Git push → Azure Pipelines triggers → build + test → publish artifact → release pipeline deploys to staging → approval gate → production.

```
# Complete .NET 8 Azure DevOps Pipeline
trigger:
  - main

pool:
  vmImage: 'windows-latest'

variables:
  solution: '**/*.sln'
  buildPlatform: 'Any CPU'
  buildConfiguration: 'Release'

steps:
# 1. Restore
- task: DotNetCoreCLI@2
  inputs:
    command: 'restore'
    projects: '$(solution)'

# 2. Build
- task: DotNetCoreCLI@2
```

```
  inputs:
    command: 'build'
    projects: '$(solution)'
    arguments: '--configuration $(buildConfiguration)'

# 3. Test + Coverage
- task: DotNetCoreCLI@2
  inputs:
    command: 'test'
    arguments: '--configuration $(buildConfiguration) --collect:"XPlat Code Coverage"'

# 4. Publish
- task: DotNetCoreCLI@2
  inputs:
    command: 'publish'
    publishWebProjects: true
    arguments: '--configuration $(buildConfiguration) --output
$(Build.ArtifactStagingDirectory)'

# 5. Upload artifact
- task: PublishBuildArtifacts@1
  inputs:
    PathtoPublish: '$(Build.ArtifactStagingDirectory)'
    ArtifactName: 'drop'
```

Q7. What deployment process are you following?

✅ **Answer:**

GitFlow branching + Azure DevOps pipelines + environment-gated releases:

- Feature branches → PR → code review → merge to develop
- Develop branch → auto-deploy to Dev environment (CI pipeline)
- Release branch → deploy to Staging → QA sign-off
- Main branch → manual approval gate → deploy to Production
- Hotfix branch → emergency pipeline → straight to Production with approval

Q8. How are you performing code coverage check?

✅ **Answer:**

```
# In Azure DevOps YAML pipeline:
- task: DotNetCoreCLI@2
  displayName: 'Run tests with coverage'
  inputs:
    command: test
    arguments: '--collect:"XPlat Code Coverage" --settings coverlet.runsettings'

- task: reportgenerator@5
  inputs:
    reports: '$(Agent.TempDirectory)/**/coverage.cobertura.xml'
    targetdir: '$(Build.ArtifactStagingDirectory)/coverage'
    reporttypes: 'HtmlInline_AzurePipelines;Cobertura'

- task: PublishCodeCoverageResults@1
  inputs:
    codeCoverageTool: Cobertura
    summaryFileLocation: '$(Agent.TempDirectory)/**/coverage.cobertura.xml'
    reportDirectory: '$(Build.ArtifactStagingDirectory)/coverage'

# coverlet.runsettings — enforce 80% minimum:
# <DataCollectionRunSettings>
#   <Threshold>80</Threshold>
# </DataCollectionRunSettings>
```

> 💡 *Pipeline fails automatically if coverage drops below the threshold — gates code quality on every PR.*

Q9. Explain CI/CD (conceptual summary)

✅ **Answer:**

Term	Full Form	What It Does
CI	Continuous Integration	Automatically build + test every commit — find bugs early
CD	Continuous Delivery	Automatically prepare release — one-click deploy to prod
CD	Continuous Deployment	Automatically deploy to production — no human gate
Pipeline	Build + Test + Deploy stages	The automation sequence from code push to running software
Artifact	Compiled output of build	The .zip/.dll package deployed to servers

Section B: YAML & Azure DevOps Structure (Q10–Q18)

Q10. YAML file and versioning in Azure Repos

✅ **Answer:**

azure-pipelines.yml lives in the root of the repository and is version-controlled alongside the code. Pipeline changes go through PR review just like application code.

```
# azure-pipelines.yml — stored at repo root
# Changes to this file are code-reviewed and versioned in Git

name: $(Build.DefinitionName)_$(Build.BuildId)

# Trigger on main and release/* branches
trigger:
  branches:
    include:
      - main
      - release/*
  paths:
    exclude:
      - README.md   # don't trigger for docs-only changes

# PR validation — run on every pull request
pr:
  branches:
    include:
      - main
  paths:
    include:
      - src/**
      - tests/**
```

Q11. Understanding the 'No Hosted Parallelism Error' in Azure DevOps

✅ **Answer:**

This error means your Azure DevOps organization has NO Microsoft-hosted parallel jobs available. Free Azure DevOps accounts have 0 free parallel jobs for public pipelines.

Cause	Solution
Free org — 0 hosted parallelism	Request free grant from Microsoft (DevOps Parallelism Request form)
Used all parallel jobs	Wait for other pipelines to finish, or buy more parallel jobs
Public project restriction	Switch to Self-Hosted Agent (free, unlimited)
Agent pool not configured	Create Agent Pool and add a self-hosted agent

Q12. Understanding Self-Hosted Agent and Agent Pools

✅ **Answer:**

A self-hosted agent is your own machine (VM, on-prem server, or container) registered with Azure DevOps to run pipeline jobs. An Agent Pool is a collection of agents.

Aspect	Microsoft-Hosted	Self-Hosted
Cost	Charged per parallel job	Free — your own machine
Setup	Zero — ready to use	Install + configure agent
Customization	Fixed tools/images	Install any tool you need
Network access	Public internet only	Can reach private VNets, on-prem DBs
Persistence	Fresh VM every run	Persistent — can cache packages
Use when	Standard builds, public repos	Private networks, custom tools, cost saving

Q13. Running, Downloading, Extracting Agents, Running Config & Entering PAT

✅ **Answer:**

Step-by-step self-hosted agent setup on Windows:

```
# Step 1: Create Agent Pool in Azure DevOps
# Organisation Settings → Agent Pools → New Pool → Self-hosted

# Step 2: Download agent
# Azure DevOps → Agent Pool → New Agent → Windows → Download

# Step 3: Extract (PowerShell as Administrator)
mkdir C:\agents\myagent
cd C:\agents\myagent
Add-Type -AssemblyName System.IO.Compression.FileSystem
[System.IO.Compression.ZipFile]::ExtractToDirectory('$HOME\Downloads\vsts-agent-win-x64-
3.x.x.zip', 'C:\agents\myagent')

# Step 4: Configure the agent
.\config.cmd
# Prompts:
# Server URL:   https://dev.azure.com/YOUR_ORG
# Auth type:    PAT
```

```
# PAT:          (paste Personal Access Token — needs Agent Pools Read & Manage)
# Agent pool:   MyPool
# Agent name:   MyAgent-01
# Work folder:  _work

# Step 5: Run as service (installs + starts Windows service)
# During config.cmd, choose: Run agent as service? Y

# Step 6: Verify — agent shows Online in Azure DevOps Agent Pool
```

Q14. Assigning Self-Hosted Agents in YAML Pipeline

✅ **Answer:**

```
# Use self-hosted agent pool by name
pool:
  name: 'MyPool'        # your pool name
  demands:
    - Agent.OS -equals Windows_NT   # optional: require Windows agent
    - dotnet -equals 8.0             # optional: require .NET 8 installed

# Or target a specific agent by name
pool:
  name: 'MyPool'
  demands:
    - Agent.Name -equals MyAgent-01

# Use hosted for CI, self-hosted for CD (access private network)
stages:
- stage: CI
  pool:
    vmImage: 'ubuntu-latest'      # Microsoft-hosted for build
- stage: CD
  pool:
    name: 'MyPool'                # Self-hosted for deploy (private DB access)
```

Q15. Running YAML Pipeline with Self-Hosted Agents

✅ **Answer:**

- Commit azure-pipelines.yml with pool: name: 'YourPool'
- Push to trigger branch → Pipeline queue appears in Azure DevOps
- Pipeline waits for an agent in the pool to be online and idle
- Agent picks up the job → runs steps on your machine
- Logs stream live in Azure DevOps → results appear in Runs tab
- If agent is offline, pipeline waits (or times out after configured limit)

Q16. Definition and Acronym of YAML

✅ **Answer:**

YAML = 'YAML Ain't Markup Language' (recursive acronym). Originally: 'Yet Another Markup Language'. It is a human-readable data serialization format used for configuration files.

Feature	Description
Full form	YAML Ain't Markup Language
File extension	.yml or .yaml
Data types	Strings, integers, booleans, lists, maps/objects, null
Key design	Indentation with spaces (NOT tabs) defines structure

Common uses	Azure Pipelines, Kubernetes manifests, Docker Compose, GitHub Actions

Q17. YAML Basics: name/value, space-indented, nested objects & hyphens

✅ **Answer:**

```
# Name/Value pairs (key: value)
name: MyPipeline
version: 1
enabled: true

# Nested objects (indented with 2 spaces)
trigger:
  branches:
    include:
      - main          # hyphen = list item
      - develop

# Inline list
tags: [build, dotnet, ci]

# Multi-line string (| = preserve newlines)
script: |
  dotnet restore
  dotnet build
  dotnet test

# Quoted strings (use when value has special chars)
message: 'Hello: World'
path: "C:\\Users\\agent"

# Booleans
isProduction: true
skipTests: false

# Null
optionalParam: ~    # or just omit the key

# COMMON MISTAKE: tabs are NOT allowed — use spaces only
# trigger:
#    branches:   ← This will FAIL — tab character
```

Q18. Understanding Azure DevOps YAML Structure — Steps and Tasks

✅ **Answer:**

```
# Full YAML hierarchy:
# Pipeline → Stages → Jobs → Steps → Tasks/Scripts

trigger: [main]

stages:
- stage: Build                    # Stage: groups related jobs
  displayName: 'Build Stage'
  jobs:
  - job: BuildJob                 # Job: runs on one agent
    displayName: 'Build .NET App'
    pool:
      vmImage: 'ubuntu-latest'
    steps:                        # Steps: ordered list of actions

    # Script step — run shell command
    - script: echo 'Starting build'
```

```
    displayName: 'Echo message'

  # Task step — predefined Azure DevOps task
  - task: UseDotNet@2          # Task name @ version
    displayName: 'Install .NET 8'
    inputs:                    # Task-specific inputs
      packageType: 'sdk'
      version: '8.0.x'

  - task: DotNetCoreCLI@2
    displayName: 'Build'
    inputs:
      command: 'build'
      projects: '**/*.csproj'
      arguments: '--configuration Release'

  # PowerShell step
  - task: PowerShell@2
    inputs:
      targetType: 'inline'
      script: Write-Host 'Custom PowerShell'
```

Section C: Artifacts, Publish & Release Pipelines (Q19–Q28)

Q19. Practicing Pipeline UI — Recent, All, Runs, Editing and Seeing Details

✅ Answer:

Azure DevOps Pipeline UI navigation:

UI Area	Where to Find	What It Shows
Recent	Pipelines → left sidebar	Last run of each pipeline with status
All	Pipelines → All tab	Every pipeline in the project
Runs	Click pipeline → Runs tab	All historical runs with status, duration, trigger
Edit	Click pipeline → Edit button	YAML editor with IntelliSense in browser
Run Details	Click any run	Each stage → job → step with live/historical logs
Test Results	Run → Tests tab	Pass/fail counts, failed test details
Code Coverage	Run → Code Coverage tab	Coverage % per class/method

Q20. Updating YAML with Publish Task

✅ Answer:

```
# Add publish task to upload build output as artifact
steps:
- task: DotNetCoreCLI@2
  displayName: 'Publish web app'
  inputs:
    command: 'publish'
    publishWebProjects: true
    arguments: '--configuration Release --output $(Build.ArtifactStagingDirectory)'
    zipAfterPublish: true          # creates .zip for easy download/deploy
```

```
# Upload the published output as a pipeline artifact
- task: PublishBuildArtifacts@1
  displayName: 'Upload artifact'
  inputs:
    PathtoPublish: '$(Build.ArtifactStagingDirectory)'
    ArtifactName: 'drop'           # artifact name used in release pipeline
    publishLocation: 'Container'  # stores in Azure DevOps

# Alternative: PublishPipelineArtifact (newer, faster)
- task: PublishPipelineArtifact@1
  inputs:
    targetPath: '$(Build.ArtifactStagingDirectory)'
    artifact: 'drop'
```

Q21. Running the pipeline and checking the artifact

✅ **Answer:**

- Push code / click 'Run pipeline' → pipeline starts automatically
- Navigate to Pipelines → your pipeline → latest Run
- Click each Stage → Job → Step to see live streaming logs
- After completion: click Artifacts tab (top right of run summary)
- Artifact 'drop' appears — click to expand and see the .zip file
- Download the artifact or use it in a Release Pipeline

Q22. Drop folder, Extracting artifact, Viewing the compiled binaries

✅ **Answer:**

```
# In release pipeline: Download artifact from build
- task: DownloadBuildArtifacts@1
  inputs:
    buildType: 'current'
    downloadType: 'single'
    artifactName: 'drop'
    downloadPath: '$(System.ArtifactsDirectory)'

# The drop folder contains the published .zip
# Default path: $(System.ArtifactsDirectory)/drop/

# Extract and view contents (PowerShell):
$zip = '$(System.ArtifactsDirectory)\drop\MyApp.zip'
Expand-Archive -Path $zip -DestinationPath 'C:\inetpub\myapp'

# Compiled binaries you'll see:
# MyApp.dll          ← compiled C# assembly
# MyApp.exe          ← self-contained executable (if configured)
# MyApp.deps.json    ← dependency metadata
# MyApp.runtimeconfig.json  ← runtime configuration
# wwwroot/           ← static files (Angular build output)
# appsettings.json   ← configuration (never put secrets here!)
```

Q23. Manual Review — Installing IIS and publishing the ASP.NET app

✅ **Answer:**

```
# Install IIS on Windows Server (PowerShell as Admin)
Install-WindowsFeature -name Web-Server -IncludeManagementTools
Install-WindowsFeature -name NET-Framework-45-ASPNET
Install-WindowsFeature -name Web-Asp-Net45

# Install .NET 8 Hosting Bundle on the server
# Download from: https://dotnet.microsoft.com/download/dotnet/8.0
```

```
# Run: dotnet-hosting-8.x.x-win.exe

# Create IIS Site (PowerShell)
Import-Module WebAdministration
New-WebSite -Name 'MyApp' -Port 80 -PhysicalPath 'C:\inetpub\myapp' -Force

# Set Application Pool to No Managed Code (for .NET Core)
Set-ItemProperty IIS:\AppPools\MyApp -Name managedRuntimeVersion -Value ''
Set-ItemProperty IIS:\AppPools\MyApp -Name processModel.identityType -Value 4

# Open firewall port
New-NetFirewallRule -DisplayName 'HTTP' -Direction Inbound -Port 80 -Protocol TCP -Action
Allow

# Test
Invoke-WebRequest http://localhost/health
```

Q24. Creating the release pipeline

✅ Answer:

Classic Release Pipeline (GUI-based, not YAML) in Azure DevOps:

- Pipelines → Releases → New Pipeline
- Select template: 'Azure App Service Deployment' or start empty
- Add Artifact — link to the build pipeline (source of the drop artifact)
- Add Stage: name it 'Staging' or 'Production'
- Add Task: 'Azure App Service Deploy' — configure subscription, app name, package path
- Set Continuous Deployment trigger — auto-deploy when new artifact available
- Add Pre-deployment Approval for Production stage

Q25. Understanding Deployment Groups

✅ Answer:

A Deployment Group is a set of physical/virtual machines registered as targets for Classic Release Pipeline deployments — used instead of Agent Pools for deploying directly to servers.

Concept	Explanation
Deployment Group	Collection of target machines (on-prem/VMs) for release deployments
Deployment Group Agent	Agent installed on each target machine — receives deployment tasks
Tag	Label on each machine — e.g. 'web', 'db' — pipelines target specific tags
vs Agent Pool	Agent Pool = runs CI jobs; Deployment Group = targets for CD to specific servers
Use case	Deploy to on-prem IIS servers, VM fleets, non-Azure servers

Q26. Creating Deployment Groups using PowerShell Scripts

✅ Answer:

```
# In Azure DevOps: Pipelines → Deployment Groups → New
# Copy the registration script (PowerShell) shown in the UI

# Run on each TARGET machine (as Administrator):
```

```
$ErrorActionPreference = 'Stop'

# The script Azure DevOps provides looks like:
$VSTSUrl    = 'https://dev.azure.com/YOUR_ORG'
$Token      = 'YOUR_PAT_TOKEN'
$TeamProject = 'YourProject'
$DeploymentGroup = 'MyDeploymentGroup'

# Download and run the agent config
[Net.ServicePointManager]::SecurityProtocol = [Net.SecurityProtocolType]::Tls12
$WebClient = New-Object System.Net.WebClient
$Uri = "$VSTSUrl/_apis/distributedtask/packages/agent"
# ... agent is installed as a Windows service on the target machine

# After running, machine appears in Azure DevOps Deployment Group
# Add tags: e.g. 'web', 'api', 'production'
```

Q27. Creating Release Pipelines Understanding Stages

✅ Answer:

Stage	Environment	Approval	Auto-Trigger
Stage 1: Dev	Development	None — auto	On every artifact
Stage 2: QA	Quality Assurance	None — auto	After Dev success
Stage 3: UAT	User Acceptance	Pre-deploy approval	After QA success
Stage 4: Prod	Production	Pre-deploy + Post-deploy approval	Manual or after UAT

> 💡 *Use environment-specific variable groups and Key Vault links per stage — never hardcode connection strings in the pipeline.*

Q28. Running the Release Pipeline and Deploying

✅ Answer:

- Release Pipeline triggers when CI builds a new artifact (or manual trigger)
- Pipelines → Releases → click the release → see all stages
- Stages run left-to-right: Dev → QA → UAT → Prod
- Pending approval stages show yellow — approver gets email/Teams notification
- Click 'Approve' → stage deploys → status turns green
- Click any stage → View logs → see each task's output
- Rollback: click '...' on deployment → Redeploy → pick previous successful deployment

Section D: Complete Azure DevOps Cycle Revision (Q29)

Q29. Revising the complete Azure DevOps Cycle

✅ Answer:

End-to-end cycle: Plan → Code → Build → Test → Release → Deploy → Monitor → Feedback.

Phase	Azure DevOps Tool	Key Actions
Plan	Boards (Epics, Stories, Tasks)	Sprint planning, backlog grooming, task assignment

Code	Repos (Git)	Feature branches, PRs, code reviews, branch policies
Build (CI)	Pipelines — YAML	Restore, build, unit test, code coverage, publish artifact
Test	Test Plans + Pipeline gates	Integration tests, UI tests, coverage thresholds
Release (CD)	Pipelines — Release	Staged deployments, approval gates, variable groups
Deploy	Deployment Groups / AKS	IIS, App Service, Kubernetes, VM deployment
Monitor	App Insights + Dashboards	Performance, errors, user flows, availability tests
Feedback	Boards + Work items	Bug reports, sprint retrospectives, backlog updates

```
# Complete cycle summary in code:

# 1. Developer creates feature branch
git checkout -b feature/add-payment

# 2. Writes code + unit tests, pushes
git push origin feature/add-payment

# 3. Creates PR → triggers CI pipeline
# CI: restore → build → test → coverage → publish artifact

# 4. PR approved → merged to develop
# CI triggers again on develop

# 5. Artifact promoted to Release Pipeline
# Stage 1 (Dev): auto-deploy
# Stage 2 (QA): auto-deploy + integration tests
# Stage 3 (UAT): approval required → manual approval
# Stage 4 (Prod): approval required → deploy

# 6. Monitor production
# App Insights alerts if error rate > 1%
# On-call engineer gets PagerDuty alert

# 7. Hotfix if needed
git checkout -b hotfix/fix-payment-bug
# Emergency pipeline → skip to Prod with approval
```

> 💡 *The entire cycle — from code commit to production — can happen in under 30 minutes with a mature Azure DevOps setup.*

CODING PROBLEMS

70 Interview Problems — C#, LINQ, SQL, JavaScript

Arrays · Strings · Recursion · Two Pointers · EF Core · SQL · System Design Coding

Section A: Array Problems (Q1–Q5, Q16–Q17, Q20, Q28–Q30, Q39–Q40, Q45)

Q1. Find two numbers in an array that add up to a target

✓ Answer:

Two-pointer or HashSet approach — O(n) time.

```
// Two Sum — HashSet O(n)
int[] FindTwoSum(int[] nums, int target) {
    var seen = new HashSet<int>();
    foreach (var n in nums) {
        int complement = target - n;
        if (seen.Contains(complement))
            return new[] { complement, n };
        seen.Add(n);
    }
    return Array.Empty<int>();
  }

  // Test:
  var result = FindTwoSum(new[] {2,7,11,15}, 9);
  // Output: [2, 7]
```

```
Output: [2, 7]
```

Q3. Reverse Array

✓ Answer:

```
// Method 1: Array.Reverse (in-place)
int[] arr = {1,2,3,4,5};
Array.Reverse(arr);
Console.WriteLine(string.Join(", ", arr)); // 5,4,3,2,1

// Method 2: Two pointer
void ReverseArray(int[] a) {
    int l=0, r=a.Length-1;
    while(l<r) { (a[l],a[r])=(a[r],a[l]); l++; r--; }
}
```

```
Output: 5, 4, 3, 2, 1
```

Q5. Move 0's to end of the array (keep order of non-zeros)

✓ Answer:

```
// Two pointer — O(n) in-place
void MoveZerosToEnd(int[] arr) {
    int insertPos = 0;
    foreach (var n in arr)
        if (n != 0) arr[insertPos++] = n;
    while (insertPos < arr.Length)
        arr[insertPos++] = 0;
  }

  int[] a = {4,0,7,5,0,2,8,0,0};
  MoveZerosToEnd(a);
```

```
  Console.WriteLine(string.Join(", ",a));
```

```
Output: 4, 7, 5, 2, 8, 0, 0, 0, 0
```

Q16. int[] GivenArray = {1,2,3,4,5,6}; Print Output: 1 2 3 4 / 5 6 1 2

Answer:

Print first 4 elements, then next 2 + wrap around first 2 (circular rotation view).

```
int[] arr = {1,2,3,4,5,6};
int n = arr.Length;

// First row: first 4 elements
Console.WriteLine(string.Join(" ", arr[..4]));  // 1 2 3 4

// Second row: elements from index 4, then wrap
var row2 = arr[4..].Concat(arr[..2]).ToArray();
Console.WriteLine(string.Join(" ", row2));       // 5 6 1 2
```

```
Output: 1 2 3 4 5 6 1 2
```

Q17. Left/Right rotate an array by K places

Answer:

```
// Left rotate by K
int[] LeftRotate(int[] arr, int k) {
    k %= arr.Length;
    return arr[k..].Concat(arr[..k]).ToArray();
  }

  // Right rotate by K
  int[] RightRotate(int[] arr, int k) {
    k %= arr.Length;
    return arr[^k..].Concat(arr[..^k]).ToArray();
  }

  var a = new[]{1,2,3,4,5};
  Console.WriteLine(string.Join(",", LeftRotate(a,2)));   // 3,4,5,1,2
  Console.WriteLine(string.Join(",", RightRotate(a,2))); // 4,5,1,2,3
```

```
Output: Left: 3,4,5,1,2  |  Right: 4,5,1,2,3
```

Q20. Find max number in Array

Answer:

```
int[] nums = {3,1,9,7,2,8};

// Method 1: LINQ
int max1 = nums.Max(); // 9

// Method 2: Manual loop
int max2 = nums[0];
foreach(var n in nums) if(n>max2) max2=n;

// Method 3: Array method
int max3 = nums.Aggregate(Math.Max);
```

```
Output: 9
```

Q28. Two pointer approach for array problems

Answer:

Two pointers (left + right) walk toward each other — O(n) instead of O(n²).

```
// Two Sum in sorted array — O(n)
(int,int) TwoSumSorted(int[] arr, int target) {
    int l=0, r=arr.Length-1;
    while(l<r) {
        int sum=arr[l]+arr[r];
        if(sum==target) return (l,r);
        else if(sum<target) l++;
        else r--;
    }
    return (-1,-1);
  }

  // Remove duplicates from sorted array in-place
  int RemoveDups(int[] arr) {
    if(arr.Length==0) return 0;
    int slow=0;
    for(int fast=1;fast<arr.Length;fast++)
        if(arr[fast]!=arr[slow]) arr[++slow]=arr[fast];
    return slow+1; // count of unique elements
  }
```

Q29. Merge two sorted arrays

Answer:

```
// Merge sort merge step — O(n+m)
int[] MergeSorted(int[] a, int[] b) {
    int i=0,j=0,k=0;
    int[] result=new int[a.Length+b.Length];
    while(i<a.Length && j<b.Length)
        result[k++]=a[i]<=b[j]?a[i++]:b[j++];
    while(i<a.Length) result[k++]=a[i++];
    while(j<b.Length) result[k++]=b[j++];
    return result;
  }

  var r=MergeSorted(new[]{1,3,5},new[]{2,4,6});
  Console.WriteLine(string.Join(",",r));
```

```
Output: 1,2,3,4,5,6
```

Q30. Find duplicate in array

Answer:

```
// HashSet — O(n) time, O(n) space
IEnumerable<int> FindDuplicates(int[] arr) {
    var seen=new HashSet<int>();
    return arr.Where(n=>!seen.Add(n)).Distinct();
  }

  // LINQ GroupBy
  var dups=new[]{1,2,3,2,4,3,5}
    .GroupBy(x=>x)
    .Where(g=>g.Count()>1)
    .Select(g=>g.Key);
  Console.WriteLine(string.Join(",",dups)); // 2,3
```

```
Output: 2, 3
```

Q39. Array moves problem — First two, Last two, First & last must all produce same sum S, return max moves

Answer:

Sliding window: find the minimum number of elements from both ends that sum to S, then calculate max moves.

```
// Each move removes A[l]+A[l+1], A[r-1]+A[r], or A[l]+A[r]
// All moves must produce same sum S
// Return: maximum number of valid moves
int MaxMoves(int[] A) {
    int n=A.Length, maxMoves=0;
    // Find all pairs from left end
    var leftSums=new Dictionary<int,int>();
    for(int i=0;i<n-1;i++) {
        int s=A[i]+A[i+1];
        leftSums.TryGetValue(s,out int c);
        leftSums[s]=c+1;
    }
    // Try each right pair
    for(int i=n-1;i>0;i--) {
        int s=A[i]+A[i-1];
        if(leftSums.ContainsKey(s))
            maxMoves=Math.Max(maxMoves,leftSums[s]+1);
    }
    return maxMoves;
  }
```

Q40. Maximize Income: Given prices [4,1,2,3] starting with one asset, max income? If answer is 2,999,999,998 what to return?

✓ Answer:

Buy low sell high — track peaks and valleys. If result > int.MaxValue (2,147,483,647), return int.MaxValue (clamping for 32-bit).

```
long MaxIncome(int[] prices) {
    long income=0;
    for(int i=1;i<prices.Length;i++)
        if(prices[i]>prices[i-1])
            income+=prices[i]-prices[i-1];
    return income;
  }

  // If result may overflow int: clamp to int.MaxValue
  int MaxIncomeClamped(int[] prices) {
    long result=MaxIncome(prices);
    return (int)Math.Min(result,(long)int.MaxValue);
  }
  // 2,999,999,998 > int.MaxValue → return 2,147,483,647
  // Time complexity: O(n) — single pass
```

```
Output: [4,1,2,3] → income = (2-1)+(3-2) = 2
```

Q45. Sort array {4,0,7,5,2,8,0,0,0} → Output: {0,0,0,0,4,7,5,2,8}

✓ Answer:

Move all zeros to front, keep non-zeros in original order (stable partition).

```
int[] arr={4,0,7,5,2,8,0,0,0};
var zeros=arr.Where(x=>x==0);
var nonZeros=arr.Where(x=>x!=0);
int[] result=zeros.Concat(nonZeros).ToArray();
Console.WriteLine(string.Join(",",result));
```

```
Output: 0,0,0,0,4,7,5,2,8
```

Section B: String Problems (Q2, Q4, Q6–Q15, Q36–Q38, Q43, Q46)

Q2. Reverse words in a string

✅ **Answer:**

```
string ReverseWords(string s)
    => string.Join(" ", s.Split(' ').Reverse());

Console.WriteLine(ReverseWords("Hello World"));  // World Hello
Console.WriteLine(ReverseWords("the sky is blue")); // blue is sky the
```

▶ **Output: World Hello**

Q4. Find occurrence of each character in a given string

✅ **Answer:**

```
string input="programming";
var freq=input.GroupBy(c=>c)
              .ToDictionary(g=>g.Key,g=>g.Count());
foreach(var kv in freq)
    Console.WriteLine($"'{kv.Key}': {kv.Value}");
```

▶ **Output: p:1, r:2, o:1, g:2, a:1, m:2, i:1, n:1**

Q6. Find the First Non-Repeating Character

✅ **Answer:**

```
char? FirstNonRepeating(string s)
    => s.GroupBy(c=>c)
        .FirstOrDefault(g=>g.Count()==1)?.Key;

Console.WriteLine(FirstNonRepeating("leetcode")); // l
Console.WriteLine(FirstNonRepeating("aabb"));     // null
```

▶ **Output: l**

Q7. Remove Duplicates from a String

✅ **Answer:**

```
// Preserve first occurrence order
string RemoveDups(string s) {
    var seen=new HashSet<char>();
    return new string(s.Where(c=>seen.Add(c)).ToArray());
  }
Console.WriteLine(RemoveDups("programming")); // progamin
```

▶ **Output: progamin**

Q8. Reverse a String

✅ **Answer:**

```
// Method 1: LINQ
string rev1=new string("Hello".Reverse().ToArray());

// Method 2: char array
string Reverse(string s){
    char[] arr=s.ToCharArray();
    Array.Reverse(arr);
    return new string(arr);
  }
Console.WriteLine(Reverse("Hello")); // olleH
```

▶ **Output: olleH**

Q9. Swapping two numbers

✅ **Answer:**

```
// Method 1: Tuple deconstruction (modern C#)
int a=5, b=10;
(a,b)=(b,a);
Console.WriteLine($"a={a}, b={b}"); // a=10, b=5

// Method 2: XOR (no temp variable)
int x=5,y=10;
x^=y; y^=x; x^=y;

// Method 3: Arithmetic
int p=5,q=10;
p=p+q; q=p-q; p=p-q;
```

▶ **Output: a=10, b=5**

Q10. Extract the surname (last word)

✅ **Answer:**

```
string name="John Michael Doe";
string surname=name.Split(' ').Last();
Console.WriteLine(surname); // Doe

// Alternative: LastIndexOf
int idx=name.LastIndexOf(' ');
string surname2=name[(idx+1)..];
```

▶ **Output: Doe**

Q11. Check if a String is a Palindrome

✅ **Answer:**

```
bool IsPalindrome(string s) {
    s=s.ToLower().Replace(" ","");
    int l=0,r=s.Length-1;
    while(l<r) if(s[l++]!=s[r--]) return false;
    return true;
  }
Console.WriteLine(IsPalindrome("racecar")); // True
Console.WriteLine(IsPalindrome("hello"));   // False
```

▶ **Output: True | False**

Q12. Find the Longest Substring Without Repeating Characters

✅ **Answer:**

Sliding window with HashSet — O(n).

```
int LongestSubstring(string s) {
    var set=new HashSet<char>();
    int max=0,l=0;
    for(int r=0;r<s.Length;r++) {
        while(set.Contains(s[r])) set.Remove(s[l++]);
        set.Add(s[r]);
        max=Math.Max(max,r-l+1);
    }
    return max;
  }
Console.WriteLine(LongestSubstring("abcabcbb")); // 3 (abc)
```

▶ **Output: 3**

Q13. Remove All White Spaces from a String

✅ Answer:

```
string s="Hello World  Foo";

// Method 1: Replace
string r1=s.Replace(" ","");  // "HelloWorldFoo"

// Method 2: LINQ
string r2=new string(s.Where(c=>c!=' ').ToArray());

// Method 3: Regex
string r3=Regex.Replace(s,@"\s+","");
```

▶ **Output: HelloWorldFoo**

Q14. Find the Longest Palindromic Substring

✅ Answer:

Expand-around-center approach — O(n²).

```
string LongestPalindrome(string s) {
    string best="";
    for(int i=0;i<s.Length;i++) {
        // Odd length (center at i)
        string odd=Expand(s,i,i);
        // Even length (center between i and i+1)
        string even=Expand(s,i,i+1);
        if(odd.Length>best.Length) best=odd;
        if(even.Length>best.Length) best=even;
    }
    return best;
  }
  string Expand(string s,int l,int r) {
    while(l>=0&&r<s.Length&&s[l]==s[r]){l--;r++;}
    return s[(l+1)..r];
  }
Console.WriteLine(LongestPalindrome("babad")); // bab or aba
```

▶ **Output: bab**

Q36. Reverse letters in each word without changing word order

✅ Answer:

```
// "Hello World" → "olleH dlroW"
string ReverseWords(string sentence)
    => string.Join(" ",
        sentence.Split(' ')
                .Select(w=>new string(w.Reverse().ToArray())));

Console.WriteLine(ReverseWords("Hello World")); // olleH dlroW
```

▶ **Output: olleH dlroW**

Q37. Count the occurrence of letters in a string

✅ Answer:

```
string s="banana";
var counts=s.GroupBy(c=>c)
            .Select(g=>$"'{g.Key}':{g.Count()}");
Console.WriteLine(string.Join(", ",counts));
// 'b':1, 'a':3, 'n':2
```

▶ **Output: 'b':1, 'a':3, 'n':2**

Q38. Date string '??-??', '?1-3?', '02-??' — return latest valid date (MM-DD format)

✓ Answer:

Greedily assign the largest valid digit to each '?' position, then validate MM (01–12) and DD (01–31).

```
string LatestDate(string pattern) {
    // pattern format: 'MM-DD' with ? as wildcard
    char[] m=pattern[..2].ToCharArray();
    char[] d=pattern[3..].ToCharArray();

    // Fill month: max valid is 12
    if(m[0]=='?') m[0]=m[1]<='2'||m[1]=='?'?'1':'0';
    if(m[1]=='?') m[0]=='1'?m[1]='2':m[1]='9';

    // Fill day: max valid is 31
    if(d[0]=='?') d[0]='3';
    if(d[1]=='?') d[0]=='3'?d[1]='1':d[1]='9';

    int month=int.Parse(new string(m));
    int day  =int.Parse(new string(d));

    if(month<1||month>12||day<1||day>31)
        return "xx-xx";

    return $"{new string(m)}-{new string(d)}";
  }
Console.WriteLine(LatestDate("??-??"));  // 19-99 → 12-31
Console.WriteLine(LatestDate("?1-3?")); // 11-39 → validate
```

Q43. Given string 'ABC', generate all combinations and permutations

✓ Answer:

```
// PERMUTATIONS
void Permute(string str,string prefix="") {
    if(str.Length==0){Console.WriteLine(prefix);return;}
    for(int i=0;i<str.Length;i++)
        Permute(str.Remove(i,1),prefix+str[i]);
  }
Permute("ABC"); // ABC,ACB,BAC,BCA,CAB,CBA

// COMBINATIONS (all non-empty subsets)
void Combine(string str,int start,string cur="") {
    if(cur.Length>0) Console.WriteLine(cur);
    for(int i=start;i<str.Length;i++)
        Combine(str,i+1,cur+str[i]);
  }
Combine("ABC",0); // A,AB,ABC,AC,B,BC,C
```

▶ **Output: Perms: ABC,ACB,BAC,BCA,CAB,CBA | Combos: A,AB,ABC,AC,B,BC,C**

Q46. Find duplicate characters in a string

✓ Answer:

```
string s="programming";
var dups=s.GroupBy(c=>c)
          .Where(g=>g.Count()>1)
          .Select(g=>g.Key);
Console.WriteLine(string.Join(",",dups)); // r,g,m
```

▶ **Output: r, g, m**

Section C: Math, Recursion & Pattern Problems (Q18–Q19, Q21–Q24, Q25, Q33–Q34)

Q18. Prime numbers — find all primes up to N

Answer:

```
// Sieve of Eratosthenes — O(n log log n)
bool[] Sieve(int n) {
    bool[] isComposite=new bool[n+1];
    for(int i=2;i*i<=n;i++)
        if(!isComposite[i])
            for(int j=i*i;j<=n;j+=i)
                isComposite[j]=true;
    return isComposite;
  }
var primes=Enumerable.Range(2,28).Where(i=>!Sieve(30)[i]);
Console.WriteLine(string.Join(",",primes));
```

```
Output: 2,3,5,7,11,13,17,19,23,29
```

Q19. Factorial using recursion

Answer:

```
long Factorial(int n) => n<=1?1:n*Factorial(n-1);
Console.WriteLine(Factorial(6)); // 720

// Iterative (no stack overflow risk for large n)
long FactIter(int n){long r=1;for(int i=2;i<=n;i++)r*=i;return r;}
```

```
Output: 720
```

Q21. Fibonacci series — 1,1,2,3,5,8,13,21

Answer:

```
// Recursive (simple, exponential time)
int Fib(int n)=>n<=1?n:Fib(n-1)+Fib(n-2);

// Iterative O(n) — preferred
void PrintFib(int count) {
    int a=0,b=1;
    for(int i=0;i<count;i++){
        Console.Write(b+" ");
        (a,b)=(b,a+b);
    }
  }
PrintFib(8); // 1 1 2 3 5 8 13 21
```

```
Output: 1 1 2 3 5 8 13 21
```

Q22. Basic Star patterns

Answer:

```
// Right triangle
for(int i=1;i<=5;i++)
    Console.WriteLine(new string('*',i));
// *
// **
// ***
// ****
// *****
```

```
// Pyramid
int n=5;
for(int i=1;i<=n;i++){
    Console.Write(new string(' ',n-i));
    Console.WriteLine(new string('*',2*i-1));
  }
//     *
//    ***
//   *****
//  *******
// *********
```

Q23. Input: ab+cde+** → Output: ((a+b)(c(d+e)))

Answer:

Postfix/reverse-polish notation to infix conversion using a stack.

```
// Postfix to Infix
string PostfixToInfix(string expr) {
    var stack=new Stack<string>();
    foreach(char c in expr) {
        if(char.IsLetter(c)) {
            stack.Push(c.ToString());
        } else {
            string b=stack.Pop();
            string a=stack.Pop();
            stack.Push($"({a}{c}{b})");
        }
    }
    return stack.Pop();
  }
Console.WriteLine(PostfixToInfix("ab+cde+**"));
// ((a+b)*(c*(d+e)))
```

```
Output: ((a+b)*(c*(d+e)))
```

Q24. What is the next number: 1, 3, 7, 13, ...

Answer:

Pattern: differences are 2, 4, 6 → next difference is 8. Answer: 13 + 8 = 21.

```
// Sequence: 1, 3, 7, 13, 21, ...
// Differences:   2, 4,  6,  8 (increases by 2)
// Formula: a(n) = a(n-1) + 2*n
// a(1)=1, a(2)=1+2=3, a(3)=3+4=7, a(4)=7+6=13, a(5)=13+8=21

int NextInSequence(int[] seq) {
    int lastDiff=seq[^1]-seq[^2];
    return seq[^1]+(lastDiff+2);
  }
Console.WriteLine(NextInSequence(new[]{1,3,7,13})); // 21
```

```
Output: 21
```

Q25. Anagram code

Answer:

```
// Two strings are anagrams if they have same chars with same frequencies
bool IsAnagram(string s,string t) {
    if(s.Length!=t.Length) return false;
    var freq=new int[26];
    foreach(char c in s) freq[c-'a']++;
    foreach(char c in t) freq[c-'a']--;
    return freq.All(f=>f==0);
  }
```

```
Console.WriteLine(IsAnagram("listen","silent")); // True
Console.WriteLine(IsAnagram("hello","world"));   // False
```

▶ **Output: True | False**

Q33. Recursive function to find factorial of a positive integer

✓ **Answer:**

```
long Factorial(int n) {
    if(n<0) throw new ArgumentException("Must be positive");
    return n<=1?1:n*Factorial(n-1);
  }

// Test:
Console.WriteLine(Factorial(0)); // 1 (0! = 1 by convention)
Console.WriteLine(Factorial(5)); // 120
Console.WriteLine(Factorial(10)); // 3628800
```

▶ **Output: 1, 120, 3628800**

Q34. Extension method: a.Add(b).Add(c) — Console.WriteLine(a.Add(b).Add(c))

✓ **Answer:**

```
// Extension method on int — enables fluent chaining
public static class IntExtensions
  {
    public static int Add(this int a, int b) => a + b;
  }

  int a=1, b=2, c=3;
  Console.WriteLine(a.Add(b).Add(c));
  // a.Add(b) = 1+2 = 3
  // 3.Add(c) = 3+3 = 6
  // Output: 6
```

▶ **Output: 6**

Q35. Single function: Add(a,b), Add(a,b,c), Add(a,b,c,d) — params keyword

✓ **Answer:**

```
// params allows variable number of arguments
int Add(params int[] numbers) => numbers.Sum();

int a=1,b=2,c=3,d=4;
Console.WriteLine(Add(a,b));     // 3
Console.WriteLine(Add(a,b,c));   // 6
Console.WriteLine(Add(a,b,c,d)); // 10

// Also works with explicit overloads if params not desired:
// int Add(int a,int b)=>a+b;
// int Add(int a,int b,int c)=>a+b+c;
```

▶ **Output: 3 | 6 | 10**

Section D: C# OOP & Language Concepts (Q15, Q26–Q27, Q44, Q47–Q56)

Q15. Boxing and Unboxing in C#

✓ **Answer:**

```
// Boxing: value type → object (heap allocation)
```

```
int value=42;
object boxed=value;  // box — allocates on heap
Console.WriteLine(boxed.GetType()); // System.Int32

// Unboxing: object → value type (explicit cast required)
int unboxed=(int)boxed;
Console.WriteLine(unboxed); // 42

// Performance impact:
var list=new List<object>(); // uses boxing for int
for(int i=0;i<1000000;i++) list.Add(i); // 1M box operations — slow!

// Avoid with generics:
var typedList=new List<int>(); // NO boxing — value type preserved
for(int i=0;i<1000000;i++) typedList.Add(i); // fast!
```

▶ **Output: System.Int32 | 42**

Q26. Remove duplicate from list

✓ **Answer:**

```
var list=new List<int>{1,2,3,2,4,3,5};

// Method 1: Distinct()
var unique=list.Distinct().ToList();
Console.WriteLine(string.Join(",",unique)); // 1,2,3,4,5

// Method 2: HashSet
var set=new HashSet<int>(list); // automatic dedup

// Method 3: GroupBy + First
var u2=list.GroupBy(x=>x).Select(g=>g.First()).ToList();
```

▶ **Output: 1,2,3,4,5**

Q44. Output: B b=new A(); b.Display(); / A a=new B(); a.Display()

✓ **Answer:**

Method hiding (new keyword) vs inheritance — compile-time type determines which method is called.

```
class A { public void Display() { Console.WriteLine('A'); } }
class B:A { public void Display() { Console.WriteLine('B'); } }

// B b = new A(); — COMPILE ERROR!
// Cannot implicitly convert A to B (A is base, B is derived)
// B is more specific than A — you can't assign A to B ref

// CORRECT: A a = new B(); — valid (upcasting)
A a=new B();
a.Display(); // Output: A — because Display() is NOT virtual
             // compile-time type is A → calls A.Display()

// If virtual + override:
// class A { public virtual void Display()... }
// class B:A { public override void Display()... }
// A a=new B(); a.Display(); → 'B' (runtime polymorphism)
```

▶ **Output: Compile Error for 'B b=new A()' | A a=new B() → 'A'**

Q47. What is Concurrent Programming?

✓ **Answer:**

```
// Concurrent: multiple tasks IN PROGRESS at the same time (may interleave)
```

```
// Parallel:   multiple tasks LITERALLY running at the same instant (multi-core)

// async/await — concurrent I/O (threads freed while waiting)
async Task RunConcurrent() {
    var t1=FetchAsync("api1");
    var t2=FetchAsync("api2");
    var results=await Task.WhenAll(t1,t2); // both run concurrently
  }

// Parallel.For — CPU parallelism (multi-core)
int count=0;
Parallel.For(0,1000,i=>Interlocked.Increment(ref count));
Console.WriteLine(count); // always 1000 — thread safe
```

Q48. What is GraphQL and why is it used?

✅ **Answer:**

Feature	REST	GraphQL
Endpoints	Multiple (/users, /orders)	Single (/graphql)
Data shape	Fixed by server	Client specifies exactly what it needs
Over-fetching	Common — extra fields returned	Eliminated — only requested fields
Under-fetching	Multiple calls needed	Single query for nested data
Versioning	URL /v1, /v2	Schema evolution — no versioning needed

Section E: SQL Coding Problems (Q59–Q66, Q69–Q70)

Q59. Find Nth highest salary

✅ **Answer:**

```
-- Method 1: DENSE_RANK (handles ties correctly)
WITH RankedSalaries AS (
    SELECT Salary,
           DENSE_RANK() OVER (ORDER BY Salary DESC) AS Rnk
    FROM Employees
  )
  SELECT DISTINCT Salary
  FROM RankedSalaries
  WHERE Rnk = @N; -- e.g. @N=2 for 2nd highest

-- Method 2: Subquery (simple, no ties handling)
SELECT MAX(Salary) FROM Employees
  WHERE Salary < (SELECT MAX(Salary) FROM Employees);
```

Q60. Count of employees having 2nd highest salary in each department

✅ **Answer:**

```
WITH DeptRanked AS (
    SELECT DepartmentID, Salary,
           DENSE_RANK() OVER(
               PARTITION BY DepartmentID
               ORDER BY Salary DESC
           ) AS Rnk
```

```
    FROM Employees
  )
  SELECT DepartmentID, Salary, COUNT(*) AS EmployeeCount
  FROM DeptRanked
  WHERE Rnk=2
  GROUP BY DepartmentID, Salary;
```

Q61. Find the Employee's Manager in the same table (self-join)

✅ **Answer:**

```
-- Self-join on same table
SELECT e.EmployeeID, e.Name AS Employee,
       m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerID=m.EmployeeID;

-- Employees with no manager (CEO)
-- LEFT JOIN ensures they appear with NULL Manager
```

Q62. COUNT(*) vs COUNT(name) vs COUNT(address) — which is faster?

✅ **Answer:**

Method	What It Counts	NULLs	Performance
COUNT(*)	All rows	Includes rows with NULLs	Fastest — no column read
COUNT(name)	Non-NULL name values	Excludes NULL names	Slower — must check name
COUNT(address)	Non-NULL addresses	Excludes NULL addresses	Slowest if address is nullable + unindexed

> 💡 *COUNT(*) is fastest — use it when you want total row count. COUNT(col) is used when you want non-NULL count of a specific column.*

Q63. Find max salary from employees

✅ **Answer:**

```
SELECT MAX(Salary) AS MaxSalary FROM Employees;

-- With department:
SELECT DepartmentID, MAX(Salary) AS MaxSalary
FROM Employees GROUP BY DepartmentID;
```

Q64. Find employees having 2nd highest salary

✅ **Answer:**

```
-- Dense rank approach:
WITH Ranked AS (
    SELECT *, DENSE_RANK() OVER(ORDER BY Salary DESC) AS Rnk
    FROM Employees
  )
  SELECT * FROM Ranked WHERE Rnk=2;
```

Q65. Left join in LINQ — method syntax and query syntax

✅ **Answer:**

```
// Method syntax — GroupJoin + SelectMany
var result=departments
```

```
    .GroupJoin(employees,
        d=>d.DeptId,
        e=>e.DeptId,
        (d,emps)=>new{d,emps})
    .SelectMany(
        x=>x.emps.DefaultIfEmpty(),
        (x,e)=>new{x.d.DeptName, EmpName=e?.Name??"No Employee"});

// Query syntax
var q=from d in departments
       join e in employees on d.DeptId equals e.DeptId into grp
       from e in grp.DefaultIfEmpty()
       select new{d.DeptName,EmpName=e?.Name??"No Employee"};
```

Q66. CREATE CLUSTERED INDEX vs CREATE NONCLUSTERED INDEX

✓ Answer:

```
-- CLUSTERED: physically sorts table rows by index key
-- Only 1 per table (usually Primary Key)
CREATE CLUSTERED INDEX IX_Employees_ID
ON Employees(EmployeeID);

-- NONCLUSTERED: separate structure with pointer to data row
-- Up to 999 per table
CREATE NONCLUSTERED INDEX IX_Employees_Dept
ON Employees(DepartmentID)
INCLUDE (Name, Salary); -- cover query with INCLUDE
```

Feature	Clustered	Non-Clustered
Count per table	1	Up to 999
Data storage	Table IS the index (leaf=data)	Separate structure, pointer to row
Speed on PK	Fastest	Fast but extra lookup
Best for	PK, range queries	Search/filter on non-PK columns

Q67. Find the 555554444333221 pattern

✓ Answer:

Pattern: 5 fives, 4 fours, 3 threes, 2 twos, 1 one. Decreasing digit count.

```
// Generate: N appears N times, from N down to 1
var sb=new System.Text.StringBuilder();
for(int i=5;i>=1;i--)
    sb.Append(new string((char)('0'+i),i));
Console.WriteLine(sb.ToString());
// Output: 555554444333221

// SQL to generate pattern:
-- WITH nums AS (SELECT number FROM master..spt_values WHERE type='P' AND number BETWEEN
1 AND 5)
-- SELECT STRING_AGG(REPLICATE(CAST(number AS CHAR),number),'') FROM nums ORDER BY number
DESC
```

```
Output: 555554444333221
```

Q69. Most occurring character in a string

✓ Answer:

```
string s="programming";
var mostFreq=s.GroupBy(c=>c)
              .OrderByDescending(g=>g.Count())
```

```
                .First();
Console.WriteLine($"'{mostFreq.Key}' appears {mostFreq.Count()} times");
```

▶ **Output: 'g' appears 2 times (r,g,m all appear 2x — first wins)**

Q70. Select department with maximum employees

✅ **Answer:**

```
-- SQL
SELECT TOP 1 DepartmentID, COUNT(*) AS EmpCount
FROM Employees
GROUP BY DepartmentID
ORDER BY EmpCount DESC;

-- LINQ
var topDept=employees
    .GroupBy(e=>e.DepartmentID)
    .OrderByDescending(g=>g.Count())
    .First();
Console.WriteLine($"Dept {topDept.Key}: {topDept.Count()} employees");
```

Section F: API, Caching & Flight Problem (Q31–Q32, Q41–Q42, Q49–Q58)

Q31. Implement caching in API

✅ **Answer:**

```
// IMemoryCache — in-process cache
[HttpGet("{id}")]
public async Task<IActionResult> GetProduct(int id,[FromServices]IMemoryCache cache){
    string key=$"product:{id}";
    if(!cache.TryGetValue(key,out Product? product)){
        product=await _repo.GetByIdAsync(id);
        if(product is not null)
            cache.Set(key,product,TimeSpan.FromMinutes(10));
    }
    return product is null?NotFound():Ok(product);
  }
```

Q32. Write async API method with proper error handling

✅ **Answer:**

```
[HttpGet("{id}")]
public async Task<ActionResult<Employee>> GetEmployee(int id) {
    try {
        if(id<=0) return BadRequest("ID must be positive");
        var emp=await _repo.GetByIdAsync(id);
        return emp is null?NotFound($"Employee {id} not found"):Ok(emp);
    }
    catch(Exception ex) {
        _logger.LogError(ex,"Error fetching employee {Id}",id);
        return StatusCode(500,"Internal server error");
    }
  }
```

Q41. Flight journey problem — maximum number of plane changes allowed

✅ **Answer:**

Classic layover problem: given a list of flights, find the maximum connections/changes possible within a time window. Typically a BFS/graph traversal.

```
// Simplified: max layovers in a journey with time constraints
// Given flights with departure airports, check valid connections

// General rule for layover validity:
// Minimum layover = 45 min (domestic) or 90 min (international)
// Max layovers = budget / minimum_flight_cost (greedy)

// Key insight: a connection is valid if
// nextFlight.Departure >= prevFlight.Arrival + layoverBuffer
```

Q42. Flight A lands at 10:30, Flight B departs same airport at 10:30 — is the plane change valid?

✅ Answer:

NO — it is NOT valid. Even if times match exactly, you need minimum connection time (typically 30–60 min) to: deplane, transit gates, re-board. Mathematically: Departure >= Arrival + minConnectionTime.

```
// Layover validity check
bool IsValidConnection(DateTime arrival, DateTime departure, int minMinutes=45) {
    return departure >= arrival.AddMinutes(minMinutes);
  }

DateTime land=new DateTime(2024,1,1,10,30,0);
DateTime depart=new DateTime(2024,1,1,10,30,0);
Console.WriteLine(IsValidConnection(land,depart,45)); // False
Console.WriteLine(IsValidConnection(land,depart,0));  // True (edge case)
```

▶ Output: False — 0 minutes is not enough connection time

Q57. Alternative for useEffect in React

✅ Answer:

```
// useEffect — side effects (most common)
useEffect(() => { fetchData(); }, [id]);

// Alternatives:
// 1. React Query — preferred for data fetching
const {data,isLoading}=useQuery({
  queryKey:['user',id],
  queryFn:()=>fetch(`/api/users/${id}`).then(r=>r.json())
  });

// 2. useLayoutEffect — sync, fires before paint
useLayoutEffect(()=>{ const w=ref.current.offsetWidth; },[]);

// 3. useMemo — derived computed value (not async)
const filtered=useMemo(()=>items.filter(x=>x.active),[items]);
```

Q58. public static string MergeStrings(string s1, string s2)

✅ Answer:

```
public static string MergeStrings(string s1,string s2){
    var sb=new System.Text.StringBuilder();
    int max=Math.Max(s1.Length,s2.Length);
    for(int i=0;i<max;i++){
        if(i<s1.Length) sb.Append(s1[i]);
        if(i<s2.Length) sb.Append(s2[i]);
    }
    return sb.ToString();
  }
Console.WriteLine(MergeStrings("abc","xyz"));   // axbycz
Console.WriteLine(MergeStrings("ab","xyzw"));   // axbyzw
```

```
Output: axbycz | axbyzw
```